Università di Genova
DIBRIS DIPARTIMENTO
DI INFORMATICA, BIOINGEGNERIA,
ROBOTICA E INGEGNERIA DEI SISTEMI

MSc Computer Science

Software Security & Engineering - Software Engineering

A serious game for High Performance Computing

Szymon Zinkowicz

Advisor: Prof. Daniele D'Agostino Examiner: Prof. Gianna Reggio

February, 2026

Università di Genova, DIBRIS
Via Opera Pia, 13
16145 Genova, Italy
https://www.dibris.unige.it/

To my father Piotr,
from whom I learned modesty, hard work, and integrity, and whose sacrifices made this possible.

To my mother Wioletta,
who always believed in me and pushed me forward.

To my girlfriend Magdalena,
who has always been patient and caring.

Abstract

High-Performance Computing (HPC) and parallel programming pose significant pedagogical challenges, requiring students to internalize abstractions at the beginning without tangible interactive experiences. Serious games facilitate intuitive understanding through engagement, surfacing misconceptions and enabling targeted instruction.

This thesis presents an educational serious game with the goal to support teachers in presenting fundamental parallel computing concepts—specifically OpenMP (shared memory)—through interactive card-sorting mechanics in web browsers. The game materializes abstract concepts through concrete manipulation: players sort cards collaboratively in multiplayer mode with private buffer zones, simulating OpenMP shared memory with thread-local storage. Students cooperate via a Web interface to minimize sorting time under constraints, learning concepts like speedup, scalability, communication overhead, multithreading, and message passing.

Implemented using the Godot Engine with web export capabilities, this work investigates both pedagogical and technical challenges inherent in the development of web-based educational multiplayer games. The development process involved the identification of a suitable cross-platform game engine, the design of extensible networking mechanisms, and iterative prototyping to balance educational goals with technical feasibility.

Key contributions include: (1) novel pedagogical mapping between card-sorting mechanics and HPC concepts derived from physical classroom experiments, (2) robust multiplayer state synchronization for educational purposes using GDSync, (3) responsive UI/UX solutions for manipulating 50+ interactive elements across screen sizes in web browsers, (4) comprehensive documentation of technical challenges and solutions regarding GDSync and multiplayer synchronization, and (5) an open-source, extensible platform for HPC education research suitable for conference publication and scientific journals.

Keywords: Serious Games, High-Performance Computing, Parallel Computing Education, OpenMP, Web-Based Game Development, Multiplayer Synchronization, Godot Engine, GDScript, Card Sorting, Educational Technology

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Professor Daniele D'Agostino, for his guidance, valuable insights, and continuous support throughout this thesis work. His much needed enthusiasm was what has put me on the rails for High-Performance Computing research. His expertise in High-Performance Computing and educational methodologies has been instrumental in shaping this research.

I am also grateful to Professor Maura Cerioli for serving as the reviewer of this thesis, steering me in right direction regarding organisational aspects and providing constructive feedback.

Special thanks to the Godot Engine community and the developers of the GDSync framework for their open-source contributions and support in resolving technical issues encountered during development.

Finally, I thank my family and friends for their encouragement and patience during the completion of this work.

Contents

Abstract **5**

Acknowledgements **7**

Chapter 1 Introduction **19**

1.1 Context and Motivation 19
1.1.1 The Challenge of Teaching High-Performance Computing . . . 19
1.1.2 Serious Games as Educational Tools 20
1.1.3 From Physical Experiments to Digital Tooling 20
1.2 Problem Statement 21
1.3 Research Objectives 22
1.3.1 Core Objectives 22
1.4 Proposed Solution: HPC Sorting Serious Game 22
1.5 Thesis Contributions 23
1.6 Thesis Organization 23

Chapter 2 Background and Literature Review **25**
2.1 Parallel Computing Fundamentals . . . 25
2.1.1 Parallel Programming Paradigms . . . 25
2.1.2 OpenMP: Shared-Memory Programming . . . 26
2.1.3 MPI: Distributed-Memory Programming . . . 27
2.1.4 Parallel Sorting Algorithms . . . 28
2.2 Serious Games in Education . . . 29
2.2.1 Definition and Characteristics . . . 29
2.2.2 Theoretical Foundations . . . 29
2.2.3 Empirical Evidence . . . 30
2.2.4 Serious Games in Computer Science . . . 31
2.2.5 Serious Games for HPC Education . . . 31
2.3 Web-Based Game Development . . . 33
2.3.1 Web Platform Advantages . . . 33
2.3.2 Web Development Challenges . . . 33
2.3.3 Game Engines for Web . . . 33
2.4 Multiplayer Game Architecture . . . 34
2.4.1 Client-Server Architecture . . . 34
2.4.2 Peer-to-Peer Architecture . . . 35
2.4.3 Web Transport Technologies . . . 35
2.4.4 State Synchronization Patterns . . . 36
2.5 Related Work . . . 37
2.5.1 Parallel Computing Educational Tools . . . 37
2.5.2 Serious Games for Computing Education . . . 38
2.5.3 Physical Teaching Methods . . . 39
2.6 Summary . . . 40

Chapter 3 Methodology **43**
3.1 Research Methodology . . . 43
3.1.1 Design Science Research Framework . . . 43
3.2 Requirements Analysis . . . 44
3.2.1 Educational Requirements . . . 44
3.2.2 Functional Requirements . . . 45
3.2.3 Core Concept . . . 47
3.3 Technology Selection . . . 48
3.3.1 Tool and Framework Evaluation Process . . . 48
3.3.2 Game Engine: Godot 4.x . . . 48
3.3.3 Programming Language: GDScript . . . 50
3.3.4 Multiplayer Framework: GDSync . . . 51
3.3.5 Networking Technology: HTTP/SSE Relay . . . 53
3.3.6 Supporting Development Toolchain . . . 54
3.3.7 Final Toolchain and Rationale Summary . . . 56
3.3.8 Development Approach . . . 56
3.4 Game Design Methodology . . . 57
3.4.1 Mapping HPC Concepts to Game Mechanics . . . 57
3.4.2 User Experience Design . . . 59
3.5 Evaluation Methodology . . . 60

3.6 Development Tools and Environment . . . 61
3.7 Summary . . . 61

Chapter 4 System Design and Architecture 63
4.1 Architectural Overview . . . 63
4.1.1 Design Principles . . . 63
4.1.2 System Layers . . . 64
4.2 Scene and Interaction Architecture . . . 64
4.2.1 Scene Lifecycle and Transition Logic . . . 64
4.2.2 Event-Driven Coordination . . . 65
4.3 Core Component Roles . . . 65
4.4 Multiplayer Architecture . . . 65
4.4.1 Network Topology . . . 65
4.4.2 State Synchronization Patterns . . . 66
4.4.3 GDSync Integration Boundary . . . 66
4.4.4 Connection Management . . . 67
4.5 Data Flow and Visibility Model . . . 67
4.6 Barrier Synchronization Architecture . . . 67
4.6.1 State Machine Design . . . 68
4.6.2 Integration Points . . . 68
4.7 Responsive UI/UX Architecture . . . 68
4.7.1 UI Layering and Overlay Isolation . . . 69
4.8 Performance and Scalability Constraints . . . 69
4.9 Summary . . . 69

Chapter 5 Implementation 71
5.1 Implementation Focus Areas . . . 71
5.2 Card Management and Sorting Correctness . . . 71
5.2.1 Card Generation and Initialization . . . 72
5.2.2 Sorting Validation . . . 72
5.2.3 Buffer Contiguity Check . . . 73
5.3 Component Reuse: CardManager Inheritance . . . 73
5.4 Multiplayer Synchronization Protocol . . . 74
5.4.1 CardState Serialization . . . 74
5.4.2 Lobby and Session Lifecycle . . . 75
5.4.3 GDSync Function Exposure . . . 75
5.4.4 State Broadcast and Reconciliation . . . 76
5.4.5 Visibility Management . . . 77
5.5 Barrier Synchronization . . . 77
5.5.1 BarrierManager State Machine . . . 78
5.5.2 Main Thread Selection . . . 78
5.5.3 Barrier State Transitions . . . 79
5.6 Web-Export Compatibility Patch . . . 79
5.6.1 Patch Application . . . 79
5.6.2 Connection Manager Signal Architecture . . . 80
5.7 Debugging and Development Observability . . . 81
5.7.1 Structured Logger Integration . . . 81

5.7.2 Runtime Variable Inspection . . . 81
5.8 Summary . . . 82

Chapter 6 Problems and Challenges 85
6.1 Overview . . . 85
6.2 Technology Selection Challenges . . . 86
6.2.1 Game Engine Trade-offs . . . 86
6.2.2 GDScript Performance Concerns . . . 86
6.3 GDSync Framework Challenges . . . 87
6.3.1 Protected Mode Blocking Issue . . . 87
6.3.2 Incomplete Documentation . . . 89
6.3.3 Web Export Signaling Patch Case Study . . . 89
6.3.4 Debugging Multiplayer Issues . . . 90
6.4 Multiplayer Synchronization Challenges . . . 91
6.4.1 Card Order Synchronization . . . 91
6.4.2 Timing and Race Conditions . . . 91
6.4.3 Different Client Views . . . 92
6.5 UI/UX Challenges . . . 93
6.5.1 Displaying Many Cards on Limited Screen Space . . . 93
6.5.2 Touch Interaction Precision . . . 93
6.5.3 Performance on Low-End Devices . . . 94
6.6 Testing Challenges . . . 95
6.6.1 Multiplayer Testing Complexity . . . 95
6.6.2 Lack of Automated Testing . . . 95
6.7 Educational Design Challenges . . . 96
6.7.1 Balancing Accuracy and Playability . . . 96
6.7.2 Assessment of Learning Effectiveness . . . 97
6.8 Lessons Learned . . . 97
6.8.1 Technical Lessons . . . 97
6.8.2 Design Lessons . . . 98
6.8.3 Process Lessons . . . 98
6.9 Summary . . . 98

Chapter 7 Results and Evaluation 101
7.1 Completed System Features . . . 101
7.1.1 Functional Features . . . 101
7.1.2 Educational Features . . . 102
7.1.3 Representative Use Cases . . . 102
7.1.4 Expected Messages Learned by Users . . . 104
7.1.5 Engagement Effects . . . 104
7.2 Technical Performance . . . 104
7.2.1 Frame Rate Performance . . . 104
7.2.2 Memory Usage . . . 105
7.2.3 Web Export Size . . . 106
7.3 Platform Compatibility . . . 106
7.3.1 Browser Compatibility . . . 106
7.3.2 Responsive Layout . . . 107

7.4 Usability Evaluation . . . 107
7.4.1 Informal User Testing . . . 107
7.4.2 Key Findings . . . 108
7.4.3 Suggested Improvements . . . 109
7.5 Educational Effectiveness . . . 109
7.5.1 Preliminary Assessment . . . 109
7.5.2 Pedagogical Value Proposition . . . 111
7.5.3 Comparison with Traditional Methods . . . 111
7.6 Comparison with Initial Requirements . . . 112
7.6.1 Requirements Fulfillment . . . 112
7.7 Known Limitations . . . 112
7.7.1 Current Limitations . . . 112
7.7.2 Scope Limitations . . . 113
7.8 Summary . . . 113

Chapter 8 Conclusion and Future Work 115
8.1 Summary of Contributions . . . 115
8.1.1 Primary Contributions . . . 115
8.2 Research Questions Revisited . . . 116
8.2.1 Addressing the Educational Problem . . . 117
8.2.2 Addressing the Technical Problem . . . 117
8.2.3 Addressing the Design Problem . . . 117
8.3 Implications . . . 118
8.3.1 For HPC Education . . . 118
8.3.2 For Serious Game Development . . . 119
8.4 Future Work . . . 120
8.4.1 Short-Term Enhancements . . . 120
8.4.2 Medium-Term Extensions . . . 121
8.4.3 Long-Term Research Directions . . . 122
8.4.4 Community and Ecosystem Development . . . 123
8.5 Limitations and Reflections . . . 124
8.5.1 Methodological Limitations . . . 124
8.5.2 Reflections on the Development Process . . . 125
8.6 Concluding Remarks . . . 125
8.7 Final Thoughts . . . 126

Bibliography 127

Appendix A User Manual 129
A.1 Introduction . . . 129
A.2 System Requirements . . . 129
A.2.1 Web Version (Primary) . . . 129
A.2.2 Desktop Version (Windows) . . . 129
A.3 Accessing the Game . . . 130
A.3.1 Web Version . . . 130
A.3.2 Desktop Version . . . 130
A.4 Getting Started . . . 130

A.4.1 Main Menu . . . 130
A.5 Single-Player Mode . . . 131
A.5.1 Starting a Single-Player Game . . . 131
A.5.2 Gameplay . . . 131
A.6 Multiplayer Mode . . . 132
A.6.1 Creating a Game . . . 132
A.6.2 Joining a Game . . . 132
A.6.3 Multiplayer Gameplay . . . 132
A.7 Settings . . . 133
A.8 Tips and Strategies . . . 133
A.8.1 Sorting Strategies . . . 133
A.8.2 Multiplayer Coordination . . . 134
A.9 Troubleshooting . . . 134
A.9.1 Common Issues and Solutions . . . 134
A.10 Educational Use . . . 135
A.10.1 For Students . . . 135
A.10.2 For Instructors . . . 135
A.11 Support and Resources . . . 136

Appendix B Code Listings 137
B.1 Source Code Repository . . . 137
B.2 Project Structure . . . 137
B.3 Key Components . . . 138
B.3.1 Card System . . . 138
B.3.2 Networking and Multiplayer . . . 139
B.3.3 Infrastructure . . . 139
B.4 Configuration . . . 139
B.4.1 Autoloads . . . 139
B.4.2 Web Export Settings . . . 140
B.5 Building and Running . . . 140
B.5.1 Development Setup . . . 140
B.5.2 Web Export . . . 140
B.5.3 Signaling Server . . . 141

Appendix C API Documentation 143
C.1 Introduction . . . 143
C.2 Signaling Server API . . . 143
C.2.1 Session Endpoints . . . 143
C.2.2 HTTP Lobby Endpoints (SSE) . . . 144
C.2.3 SSE Event Types . . . 144
C.2.4 WebSocket Endpoints . . . 145
C.2.5 Utility Endpoints . . . 145
C.2.6 Error Codes . . . 145
C.3 Server Data Models . . . 145
C.4 GDScript Game Classes . . . 146
C.4.1 CardManager (card_manager.gd) . . . 146
C.4.2 MultiplayerCardManager (multiplayer_card_manager.gd) . . . 147

C.4.3 BarrierManager (barrier_manager.gd) 147
C.4.4 ConnectionManager (connection_manager.gd) 148
C.5 GDSync Framework Usage 149

Appendix D Study Materials **151**
D.1 Introduction 151
D.2 Learning Objectives 151
D.2.1 Foundational Concepts 151
D.2.2 Sequential Execution Concepts (Single-Player Mode) 151
D.2.3 OpenMP Concepts (Multiplayer Mode) 152
D.3 Discussion Questions 152
D.3.1 Pre-Game Questions 152
D.3.2 Post-Game Discussion: Single-Player Mode 153
D.3.3 Post-Game Discussion: Multiplayer Mode 153
D.3.4 Connecting to HPC Concepts 153
D.4 Classroom Activities 154
D.4.1 Activity 1: Strategy Competition 154
D.4.2 Activity 2: Does More Players Mean Faster? 154
D.4.3 Activity 3: Working Together vs. Working Separately 155
D.5 Exercises 155
D.5.1 Exercise 1: Comparing Single and Multiple Players 156
D.5.2 Exercise 2: Finding the Best Team Size 156
D.5.3 Exercise 3: Dividing the Work 157
D.6 Conceptual Mappings 157
D.6.1 Game to OpenMP 157
D.7 Assessment Rubric 157
D.7.1 Conceptual Understanding (40 points) 158
D.7.2 Practical Application (30 points) 158
D.7.3 Analysis and Reflection (30 points) 158
D.8 Additional Resources 158
D.8.1 Recommended Readings 158
D.8.2 Online Resources 158
D.8.3 Hands-On Practice 159
D.9 Tool and Framework Reference Links 159
D.10 Instructor Notes 160
D.10.1 Integration with Curriculum 160
D.10.2 Common Student Misconceptions 160

List of Figures

1.1 Transformation from physical classroom activity to digital serious-game-based educational tool, preserving pedagogical effectiveness while overcoming physical limitations 21

3.1 Comparison of game modes: Single-player represents sequential execution (left) where one player sorts all cards independently; Multi-player represents OpenMP shared-memory parallelism (right) where multiple players collaborate with a shared container and private buffers 47

3.2 HTTP/SSE relay message flow: connection, lobby creation, and gameplay synchronization 55

4.1 HTTP/SSE relay-oriented architecture used for web-first multiplayer synchronization 66

5.1 VarTree debug panel showing GDSync runtime variables: Debug ID, clientID, IAmHost flag, currentLobbyID, and Players count 82

6.1 GDSync's Protected Mode toggle in the plugin settings. When enabled (default), it blocks remote calls on Nodes that haven't been explicitly exposed via `GDSync.expose_func()` or `GDSync.expose_var()` 88

List of Tables

2.1 HPC concepts mapped to game mechanics 32

3.1 Game engine comparison for HPC Sorting Game 49

3.2 Final toolchain decisions and accepted trade-offs 56

3.3 Systematic mapping of HPC concepts to game mechanics 58

5.1 Key source file locations 83

7.1 Implemented pedagogical features 103

7.2 Frame rate performance by browser 105

7.3 Memory usage by game state 105

7.4 Browser compatibility 106

7.5 Perceived advantages over traditional methods 111
7.6 Requirements fulfillment status 112

C.1 Session management endpoints 143
C.2 HTTP lobby endpoints 144
C.3 SSE event types 144
C.4 WebSocket endpoints 145
C.5 Utility endpoints 145
C.6 Signaling server error codes 146

D.1 Detailed game-to-OpenMP concept mapping 157
D.2 External tools and framework links used in this thesis 159

Listings

2.1 Simple OpenMP parallel loop 27
2.2 Simple MPI message passing 27
4.1 Architectural pattern: host-authoritative move confirmation 66
4.2 Architectural boundary: sync wrapper delegates to game manager . . 67
4.3 Architectural mechanism: private buffer visibility gate 67
5.1 Card manager initialization pipeline (card_manager.gd) 72
5.2 Sorting validation (card_manager.gd, line 645) 72
5.3 Buffer contiguity validation (card_manager.gd, line 693) 73
5.4 Multiplayer extension declaration (multiplayer_card_manager.gd, line 1) 73
5.5 Multiplayer initialization with host/client branching (multiplayer_card_manager.gd) 74
5.6 CardState transport protocol (multiplayer_card_manager.gd, lines 6–49) 74
5.7 GDSync function exposure (multiplayer_card_manager.gd) 75
5.8 Card movement broadcast (multiplayer_card_manager.gd) 76
5.9 Remote card movement handler (multiplayer_card_manager.gd) . . . 76
5.10 Buffer entry notification (multiplayer_card_manager.gd) 77
5.11 BarrierManager state machine (barrier_manager.gd) 78
5.12 Main thread determination (barrier_manager.gd) 78
5.13 Barrier state transitions (barrier_manager.gd) 79
5.14 GDSync web patch (gdsync_web_patch.gd) 79
5.15 GDSync signal wiring (connection_manager.gd) 80
5.16 Logger instantiation pattern 81
5.17 VarTree debug variable mounting (card_manager.gd) 81
6.1 Card order synchronization approach 91
C.1 GDSync function exposure (actual API) 149

Chapter 1

Introduction

Terminology Note Throughout this thesis, the artifact is referred to as a **game**. For pedagogical precision, it can also be categorized as a **serious game** and used as an instructional tool in educational contexts. This terminology aligns with prior HPC educational activities explicitly framed as serious games [1].

1.1 Context and Motivation

1.1.1 The Challenge of Teaching High-Performance Computing

High-Performance Computing (HPC) has become an indispensable tool in modern computational science, powering everything from weather forecasting and molecular dynamics simulations to machine learning and big data analytics. As computational problems grow in scale and complexity, the ability to write efficient parallel programs has transitioned from a specialized skill to a fundamental competency for software engineers and computational scientists.

However, teaching parallel computing concepts presents unique pedagogical challenges. Students frequently find it difficult to visualize how multiple threads or processes interact, communicate, and coordinate to solve problems efficiently. The principal challenges include:

- **Abstraction Gap**: Concepts like threads, processes, synchronization, and message passing are inherently abstract and difficult to visualize
- **Cognitive Load**: Understanding parallel algorithms requires simultaneous consideration of multiple execution flows, shared resources, and synchronization primitives
- **Limited Immediate Feedback**: Traditional HPC assignments involve slow compile-submit-debug cycles that impede learning
- **Language Barriers**: HPC programming typically requires C/C++ or Fortran—languages with steep learning curves that can overshadow conceptual

understanding. Students often struggle more with template errors and memory management than with parallelism itself due to the focus of C++ on keeping backwards compatibility as first priority

- **High Entry Barrier**: Setting up HPC environments and debugging distributed systems requires significant technical expertise that distracts from core concepts

Traditional lecture-based approaches struggle to convey the dynamic, interactive nature of parallel processes, motivating the exploration of alternative pedagogical approaches.

1.1.2 Serious Games as Educational Tools

Serious games—games designed with a primary purpose beyond entertainment—have emerged as powerful educational tools. By leveraging mechanics such as immediate feedback, progressive challenges, and interactive exploration, serious-game-based tools can make complex concepts more accessible and engaging.

In this thesis, the artifact is intentionally presented as a **game**. At the same time, for pedagogical precision, it can be categorized as a **serious game** and used as an instructor-guided educational tool in classroom or workshop settings.

- **Active Learning**: Learners engage directly with concepts through interaction
- **Immediate Feedback**: Actions produce visible outcomes quickly
- **Visualization**: Abstract parallel behaviors become tangible
- **Safe Experimentation**: Risk-free environment for trial and error without costly HPC infrastructure
- **Lower Entry Barrier**: Learners can focus on concepts before full programming complexity

1.1.3 From Physical Experiments to Digital Tooling

The pedagogical approach underlying this thesis stems from classroom experiments conducted by Professor Daniele D'Agostino, where students physically sorted numbered cards to simulate parallel computing paradigms:

- **Sequential Execution**: A single student sorts all cards alone—demonstrating traditional sequential algorithms
- **OpenMP Simulation**: Multiple students work collaboratively at a shared desk with all cards visible in a common container, using private buffer areas for local sorting—demonstrating shared memory parallelism with thread-local storage

These physical activities were highly effective at conveying parallel computing concepts in a tangible, memorable way. However, they faced significant limitations, including scalability (limited by desk size).

This thesis aims to capture that pedagogical effectiveness in a scalable, digital format accessible via web browsers, enabling students worldwide to experience similar learning benefits without the constraints of physical classrooms.

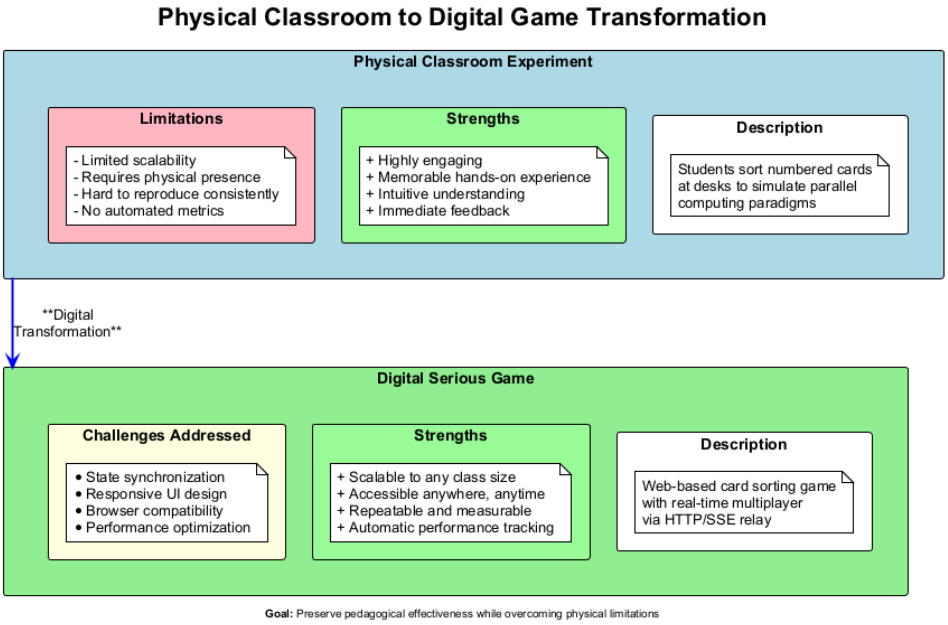

Figure 1.1: Transformation from physical classroom activity to digital serious-game-based educational tool, preserving pedagogical effectiveness while overcoming physical limitations

1.2 Problem Statement

This thesis addresses the following research problem:

> *How can we design and implement an effective web-based serious-game educational tool that teaches fundamental High-Performance Computing concepts (specifically OpenMP shared-memory parallelism and sequential vs. parallel execution) through interactive card-sorting while overcoming technical challenges of multiplayer state synchronization and responsive UI/UX constraints?*

This overarching problem decomposes into three interrelated challenges:

- **Educational Challenge**: How can abstract parallel computing concepts be mapped to concrete mechanics that accurately represent OpenMP paradigms?

- **Technical Challenge**: How can real-time multiplayer interaction be implemented across diverse devices with acceptable latency and consistent state?

- **Design Challenge**: How can we balance educational effectiveness with engagement and playability across different screen sizes and devices?

Detailed requirements analysis appears in Chapter 3.

1.3 Research Objectives

The primary objective of this thesis is to develop a functional serious-game-based educational tool that demonstrates the feasibility of teaching HPC concepts through web-based interactive activities, creating an open-source platform suitable for educational use and future research.

1.3.1 Core Objectives

1. **Develop an Interactive Game-like Educational Tool**: Use card sorting as a metaphor for parallel sorting and OpenMP-style collaboration

2. **Implement Web-First Functionality**: Run directly in modern browsers with responsive design

3. **Create Multiplayer Capability**: Support many collaborative participants with synchronized shared state

4. **Evaluate Technical Feasibility**: Assess architecture decisions and implementation challenges, including GDSync integration and HTTP/SSE relay networking

5. **Support Instructor-Guided Use**: Ensure the artifact fits classroom and workshop workflows with facilitation and reflection

1.4 Proposed Solution: HPC Sorting Serious Game

This thesis presents the **HPC Sorting Serious Game**, a web-first serious-game-based educational tool for introducing parallel computing concepts through interactive card sorting. The card interaction model provides an intuitive bridge between concrete manipulation and abstract HPC concepts. Technology selection process, alternatives explored, and final rationale are presented in Chapter 3.

1.5 Thesis Contributions

This thesis contributes to HPC education and educational game engineering in four areas:

1. **Pedagogical Contribution**: A novel game-based approach for teaching OpenMP shared-memory parallelism and sequential vs. parallel execution comparison, directly inspired by successful physical classroom experiments, with systematic mapping between game mechanics and parallel computing paradigms. Additionally, demonstrates that fundamental HPC concepts can be taught effectively without requiring students to first master complex programming languages like C/C++, reducing cognitive load.
2. **Technical Contribution**: A web-first implementation with synchronized multiplayer behavior and responsive UI for high-card-count interaction
3. **Documentation Contribution**: Detailed discussion of practical engineering issues, including synchronization and framework constraints
4. **Open-Source Contribution**: An extensible codebase that can be reused and expanded for future HPC education research

Detailed discussion of these contributions appears in Chapter 8.

1.6 Thesis Organization

The remainder of this thesis is organized as follows:

Chapter 2 reviews parallel computing fundamentals, serious games in education, web development constraints, and multiplayer architecture patterns.

Chapter 3 describes the research method, requirements analysis, and technology selection.

Chapter 4 presents system architecture, component design, networking model, and UI/UX strategy.

Chapter 5 details implementation choices and key engineering decisions.

Chapter 6 analyzes development challenges and lessons learned.

Chapter 7 presents system capabilities, technical performance, and preliminary educational observations.

Chapter 8 summarizes contributions and outlines future work.

Appendices provide supplementary materials including user documentation and technical references.

Chapter 2

Background and Literature Review

This chapter provides the foundational knowledge and contextual background necessary to understand the HPC Sorting Serious Game. We review key concepts in parallel computing, serious games in education, web-based game development, and multiplayer networking architectures.

2.1 Parallel Computing Fundamentals

Parallel computing is the simultaneous execution of multiple computational tasks to solve problems more efficiently than sequential processing. As Moore's Law approaches its physical limits, parallel computing has become essential for improving computational performance across scientific computing, data analytics, and real-time systems.

2.1.1 Parallel Programming Paradigms

Two primary paradigms dominate parallel computing education and practice: shared-memory parallelism and distributed-memory parallelism.

2.1.1.1 Shared-Memory Parallelism

In shared-memory systems, multiple processors or cores access a common address space. All threads can read and write to the same memory locations, enabling efficient data sharing but requiring careful synchronization to prevent race conditions.

Characteristics:

- All processors access the same physical memory
- Communication occurs through shared variables
- Synchronization primitives (locks, barriers, atomic operations) prevent conflicts
- Lower communication overhead compared to distributed memory

- Limited scalability due to memory bandwidth constraints

Applications: Shared-memory parallelism is ideal for problems requiring frequent communication between processing units, such as iterative algorithms, graph processing, and data analytics on multicore processors.

2.1.1.2 Distributed-Memory Parallelism

In distributed-memory systems, each processor has its own local memory. Processors communicate by explicitly sending and receiving messages over a network, requiring programmers to manage data distribution and communication patterns.

Characteristics:

- Each processor has private memory space
- Communication requires explicit message passing
- Scalable to thousands or millions of processors
- Higher communication overhead and latency
- No shared state eliminates race conditions but complicates programming

Applications: Distributed-memory parallelism is essential for large-scale scientific simulations, weather modeling, molecular dynamics, and big data processing across cluster and supercomputer architectures.

2.1.2 OpenMP: Shared-Memory Programming

OpenMP (Open Multi-Processing) is a widely-used API for shared-memory parallel programming in C, C++, and Fortran. It uses compiler directives (pragmas) to specify parallel regions, enabling incremental parallelization of sequential code.

Core Concepts:

Parallel Regions Code sections executed by multiple threads simultaneously

Work-Sharing Constructs Distribute loop iterations or tasks among threads

Data Scoping Variables can be shared (visible to all threads) or private (thread-local copies)

Synchronization Barriers, critical sections, and atomic operations coordinate thread execution

Scheduling Strategies (static, dynamic, guided) control how iterations are assigned to threads

```
#pragma omp parallel for schedule(dynamic)
for (int i = 0; i < N; i++) {
    array[i] = compute_value(i);
}
```

Listing 2.1: Simple OpenMP parallel loop

Educational Value: OpenMP's simplicity makes it excellent for teaching parallel programming concepts. The incremental approach—starting with sequential code and adding pragmas—allows students to observe immediate performance improvements while learning about thread management, data dependencies, and race conditions.

2.1.3 MPI: Distributed-Memory Programming

Example Code:

> **Note:** This section provides background on MPI for completeness and context within the broader HPC landscape. **The game implemented in this thesis focuses exclusively on OpenMP shared-memory parallelism.** MPI distributed-memory concepts are discussed here for educational context but are **not implemented** in the current version. See Chapter 8 for discussion of MPI as potential future work.

MPI (Message Passing Interface) is the de facto standard for distributed-memory parallel programming. It provides a rich set of communication primitives for sending and receiving messages between processes, enabling scalable parallel computing on clusters and supercomputers.

Core Operations:

Point-to-Point Communication `MPI_Send` and `MPI_Recv` exchange messages between specific processes

Collective Communication Operations like `MPI_Bcast`, `MPI_Gather`, and `MPI_Reduce` involve all processes in a communicator

Process Groups Communicators organize processes and define communication contexts

Data Types MPI supports both primitive and user-defined data types for message content

```
MPI_Init(&argc, &argv);
int rank, size;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```

if (rank == 0) {
    // Master process
    MPI_Send(data, count, MPI_INT, 1, 0, MPI_COMM_WORLD);
} else if (rank == 1) {
    // Worker process
    MPI_Recv(data, count, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
}

MPI_Finalize();
```

Listing 2.2: Simple MPI message passing

Educational Value: MPI teaches students about distributed computing challenges including data distribution, explicit communication, latency management, and scalability. The message-passing model forces programmers to think carefully about data ownership and communication patterns, skills essential for modern distributed systems.

2.1.4 Parallel Sorting Algorithms

Example Code: Sorting is a fundamental algorithmic problem that benefits significantly from parallelization. Several parallel sorting strategies exist, each with different communication and synchronization requirements.

Parallel Merge Sort:

- Recursively divide array into subarrays
- Sort subarrays in parallel
- Merge sorted subarrays (potentially in parallel)
- Well-suited for shared-memory systems
- Time complexity: $O(\frac{n \log n}{p})$ with p processors

Sample Sort (MPI Pattern):

- Distribute data across processes
- Each process sorts local data
- Select sample elements to determine global pivots
- Redistribute data based on pivots
- Each process sorts its final partition

- Commonly used in distributed-memory systems
- *Note: Not implemented in this thesis; discussed for context*

Relevance to Educational Game: The card-sorting game mechanics map to parallel sorting algorithms. In the implemented game, players act as threads (OpenMP) coordinating through shared memory to sort cards efficiently, making abstract algorithmic concepts tangible through gameplay. Future extensions could explore MPI-style distributed sorting (see Chapter 8).

2.2 Serious Games in Education

Serious games—games designed primarily for purposes beyond entertainment—have emerged as powerful educational tools. This section reviews the theoretical foundations and empirical evidence supporting game-based learning.

2.2.1 Definition and Characteristics

Zyda [2] defines serious games as games that "do not have entertainment, enjoyment, or fun as their primary purpose." These games leverage game design principles to achieve learning objectives, skill development, or behavioral change.

Key Characteristics:

- **Clear Learning Objectives**: Explicit educational goals aligned with curricula
- **Engaging Mechanics**: Game elements (challenges, rewards, feedback) maintain motivation
- **Active Learning**: Players learn by doing, not passive observation
- **Immediate Feedback**: Instant consequences of actions reinforce learning
- **Safe Experimentation**: Risk-free environment for trial and error
- **Progressive Difficulty**: Challenges scale with player skill development

2.2.2 Theoretical Foundations

Game-based learning draws on several established educational theories:

2.2.2.1 Constructivism

Constructivist theory posits that learners actively construct knowledge through experience rather than passively receiving information. Games naturally support constructivist learning by placing players in problem-solving scenarios where they discover principles through experimentation.

2.2.2.2 Flow Theory

Csikszentmihalyi [3] describes "flow" as the optimal psychological state where challenge level matches skill level, producing deep engagement. Well-designed educational games maintain flow by dynamically adjusting difficulty, keeping learners in the zone of proximal development.

2.2.2.3 Experiential Learning

Kolb's experiential learning cycle—concrete experience, reflective observation, abstract conceptualization, active experimentation—aligns perfectly with game mechanics. Players experience situations, observe outcomes, form hypotheses, and test them iteratively.

2.2.3 Empirical Evidence

Research consistently demonstrates the effectiveness of serious games for learning:

Connolly et al. [4] Meta-Analysis: A systematic literature review of 129 papers found that serious games positively impact learning outcomes across multiple domains. Key findings include:

- Improved knowledge acquisition and retention
- Enhanced perceptual and cognitive skills
- Increased motivation and engagement
- Better affective and motivational outcomes
- Most effective when integrated with traditional instruction

Papastergiou [5] Computer Science Education: A study of digital game-based learning in high school computer science found that students using educational games demonstrated:

- Significantly higher knowledge acquisition

- Greater enjoyment and motivation
- Preference for game-based learning over traditional methods
- Improved attitudes toward computer science

2.2.4 Serious Games in Computer Science

Several successful serious games target computer science education:

CodeCombat:

- Teaches programming through RPG-style gameplay
- Players write real code to control characters
- Supports Python, JavaScript, and other languages
- Used in thousands of classrooms worldwide

Lightbot:

- Introduces programming concepts through puzzle-solving
- Teaches sequencing, loops, and procedures
- No syntax required—visual programming interface
- Suitable for young learners

Parallel Computing Games: Few games specifically target parallel computing education. Notable exceptions include:

- **PARMACS Simulator**: Interactive visualization of parallel algorithms
- **ParSim**: Educational tool for parallel algorithm design
- **CUDA Learning Games**: GPU programming challenges

However, these tools typically focus on visualization rather than hands-on, game-like interaction, and few provide multiplayer experiences.

2.2.5 Serious Games for HPC Education

High-Performance Computing education faces unique challenges that serious games can address:

Why Games for HPC?

1. **Abstract Concepts**: HPC involves invisible processes (threads, messages, synchronization) that are difficult to visualize through static diagrams
2. **Temporal Dynamics**: Parallelism involves timing, race conditions, and coordination that unfold over time—games naturally model dynamic systems
3. **Scale Mismatch**: Real HPC systems involve millions of operations per second; games can slow down and make these concepts observable
4. **Hands-On Experience**: Students learn parallelism best by experiencing coordination challenges firsthand

Mapping HPC Concepts to Game Mechanics: The following mappings enable intuitive understanding of HPC concepts through gameplay (see Table 2.1).

Table 2.1: HPC concepts mapped to game mechanics

HPC Concept	Game Mechanic
Threads/Processes	Players
Shared Memory	Shared game board/container
Thread-Local Storage	Private player inventory
Synchronization	Turn-taking or barriers
Race Conditions	Conflicting simultaneous actions
Speedup	Time comparison (1 player vs. N players)
Communication Overhead	Time to transfer items between players
Load Balancing	Equal distribution of work

Advantages of Game-Based HPC Learning:

- **No Prerequisites**: Students can experience parallel concepts before learning programming syntax
- **Misconception Surfacing**: Games reveal incorrect mental models (e.g., "more threads always faster")
- **Quantifiable Learning**: Performance metrics (time, moves) provide concrete feedback
- **Collaborative Learning**: Multiplayer games naturally create discussion and peer learning
- **Scalable Delivery**: Web-based games reach students regardless of institutional HPC infrastructure

2.3 Web-Based Game Development

Web browsers provide a universal platform for educational game deployment, eliminating installation barriers and enabling access from any device with a modern browser.

2.3.1 Web Platform Advantages

Accessibility: Web games require no installation—students access the game via URL, eliminating software distribution challenges and platform compatibility concerns.

Cross-Platform: A single web build runs on Windows, macOS, Linux, and even mobile browsers, maximizing reach with minimal development overhead.

Instant Updates: Deploying updates is immediate—no app store approval processes or user-initiated updates required.

Low Barrier: Modern browsers include powerful JavaScript engines, WebGL for graphics, and APIs for real-time communication.

2.3.2 Web Development Challenges

Browser Sandboxing: Security restrictions prevent raw socket access, UDP communication, and local peer discovery—complicating multiplayer networking.

Performance Overhead: WebAssembly and JavaScript have overhead compared to native code, though performance is acceptable for 2D games.

Networking Limitations: Traditional peer-to-peer networking (like WebRTC data channels) can be complex to configure and may not work reliably across all network configurations.

2.3.3 Game Engines for Web

Godot Engine emerged as the optimal choice for this project due to:

- Completely free and open-source (MIT license)
- Excellent 2D rendering performance
- Lightweight web exports (smaller than Unity or Unreal)
- GDScript language similar to Python (accessible for students)
- Active community and extensive documentation

- Built-in HTML5 export templates
- Scene-based architecture promotes modular design

2.4 Multiplayer Game Architecture

Multiplayer functionality is essential for simulating OpenMP shared-memory parallelism with multiple players collaborating on shared data. This section reviews common multiplayer architectures and networking technologies.

2.4.1 Client-Server Architecture

Traditional multiplayer games use a client-server model:

Characteristics:

- Dedicated server hosts game state
- Clients send inputs to server
- Server validates actions, updates state, broadcasts to clients
- Authoritative server prevents cheating
- Requires server infrastructure and maintenance

Advantages:

- Strong consistency and cheat prevention
- Clients need not communicate directly
- Server can enforce rules and validate actions

Disadvantages:

- Requires dedicated servers (cost, maintenance)
- Single point of failure
- Latency for all communications
- Scalability challenges with many concurrent games

2.4.2 Peer-to-Peer Architecture

Peer-to-peer (P2P) networks eliminate dedicated servers by allowing clients to communicate directly:

Characteristics:

- Clients connect directly to each other
- No central server required (except for matchmaking)
- One peer often designated as "host" for authority
- Lower latency for direct connections
- More complex connection establishment (NAT traversal)

Advantages:

- No server infrastructure costs
- Lower latency between peers
- Decentralized—no single point of failure
- Scalable for small groups (2–8 players)

Disadvantages:

- NAT traversal complexity
- Host has performance burden
- Cheat prevention more difficult
- Connection quality varies with peer networks

2.4.3 Web Transport Technologies

Web browsers offer several options for real-time communication, each with trade-offs:

HTTP Polling:

- Client repeatedly requests updates from server
- Simple to implement, universally supported
- Higher latency and server load due to frequent requests
- Suitable for low-frequency updates

Server-Sent Events (SSE):

- Server pushes events to client over HTTP
- Simpler than WebSocket (HTTP-based, no special protocol)
- Unidirectional (server to client only)
- Automatic reconnection built into browser API
- Good browser support and proxy compatibility

HTTP + SSE Relay Architecture: For web-based multiplayer games requiring broad browser compatibility, a relay server architecture using HTTP POST for client-to-server communication and SSE for server-to-client events provides:

- Universal browser support without WebSocket complexity
- Works through firewalls and proxies that block WebSocket
- Simplified connection establishment (no STUN/TURN required)
- Trade-off: All traffic routes through server (no P2P)
- Acceptable latency for turn-based or slower-paced games

2.4.4 State Synchronization Patterns

Maintaining consistent game state across multiple clients is challenging. Common patterns include:

Deterministic Lockstep:

- All clients simulate the same game logic
- Inputs synchronized and executed simultaneously
- Guarantees identical state on all clients
- Sensitive to latency and packet loss
- Used in RTS games (Age of Empires, StarCraft)

Client-Side Prediction:

- Clients predict results of actions immediately
- Server validates and corrects if necessary
- Provides responsive feel despite latency
- Requires reconciliation when predictions wrong
- Used in FPS games (Counter-Strike, Overwatch)

Authoritative Server/Host:

- Server/host maintains canonical state
- Clients send actions, receive state updates
- Simple to implement and reason about
- Higher latency but stronger consistency
- Suitable for turn-based or slower-paced games

Selected Approach: For the HPC Sorting Game, a **host-authoritative model** was chosen because:

- Educational focus prioritizes clarity over competitive performance
- Collaborative nature tolerates moderate latency
- Simpler implementation reduces development complexity
- Prevents inconsistencies and ensures fair learning environment
- Aligns with centralized coordination patterns in parallel systems

2.5 Related Work

This section reviews existing educational tools and games related to parallel computing and serious games.

2.5.1 Parallel Computing Educational Tools

Several tools exist for teaching parallel computing, each with strengths and limitations:

PARMACS:

- Parallel algorithm visualization tool
- Shows thread/process execution timelines
- Limited interactivity—primarily observational
- Desktop-only platform

OpenMP Visualization Tools:

- Intel VTune, TAU (Tuning and Analysis Utilities)
- Focused on performance profiling
- Steep learning curve for students
- Not designed for introductory education

MPI Tools:

- MPICH, Open MPI with debugging tools
- Require cluster access or virtual machines
- Focus on actual parallel programming, not conceptual learning
- *Note: MPI concepts discussed for context; not covered in this thesis*

Gap Identification: Existing tools generally fall into two categories:

1. **Visualization tools**: Show parallel execution but lack hands-on interaction
2. **Programming environments**: Require coding skills, deterring beginners

Few tools bridge the gap between conceptual understanding and practical implementation. Fewer still are designed for web platforms or incorporate game mechanics to enhance engagement.

2.5.2 Serious Games for Computing Education

Several serious games successfully teach computing concepts:

CodeCombat:

- Teaches programming through RPG gameplay
- Players write real code in Python/JavaScript
- Excellent for syntax and basic algorithms
- Does not address parallel computing

Human Resource Machine:

- Puzzle game teaching assembly-like programming
- Visual programming—no syntax barriers
- Excellent for teaching sequential thinking
- Single-threaded—no parallelism concepts

Shenzhen I/O:

- Hardware design and programming game
- Teaches low-level optimization
- Complex and targeted at experienced programmers
- No explicit parallel computing focus

Gap in Parallel Computing Games: No widely-adopted serious game specifically teaches parallel computing concepts like OpenMP shared-memory parallelism through interactive, game-based mechanics on web platforms. This gap motivated the development of the HPC Sorting Serious Game, which focuses on OpenMP concepts with potential for future MPI extensions.

2.5.3 Physical Teaching Methods

The pedagogical approach for this thesis draws inspiration from physical classroom activities:

CS Unplugged:

- Collection of unplugged computer science activities
- Teaches algorithms, sorting, searching without computers
- Highly engaging but not scalable or repeatable
- Limited to in-person classroom settings

Professor D'Agostino's Card Sorting Experiments: As described in Chapter 1, Professor Daniele D'Agostino conducted classroom experiments where students physically sorted numbered cards to simulate parallel computing paradigms. These experiments demonstrated:

- High student engagement and retention
- Intuitive understanding of parallelism concepts
- Memorable hands-on experience
- Clear differentiation between sequential and parallel execution models

However, physical experiments have limitations:

- Require physical classroom presence
- Not scalable to large classes
- Difficult to reproduce consistently
- Limited flexibility in parameters (card count, student count)
- No performance metrics or automated feedback

Digital Transformation: This thesis aims to capture the pedagogical effectiveness of physical card-sorting experiments in a digital, web-first serious game that overcomes the limitations of physical activities while preserving their educational benefits.

2.6 Summary

This chapter established the foundational knowledge necessary to understand the HPC Sorting Serious Game:

- **Parallel Computing**: Reviewed OpenMP shared-memory parallelism (the focus of this thesis) and MPI distributed-memory paradigms (for context), highlighting their educational challenges and importance.
- **Serious Games**: Examined theoretical foundations, empirical evidence, and successful examples of game-based learning in computer science education.
- **Web Development**: Discussed advantages and challenges of web platforms for educational games, and the rationale for selecting Godot Engine with web export capabilities.
- **Multiplayer Architecture**: Compared client-server and peer-to-peer models, introduced web transport technologies (HTTP/SSE), and justified the host-authoritative approach for OpenMP simulation.
- **Related Work**: Identified gaps in existing parallel computing educational tools and serious games, motivating the development of this project.

The next chapter describes the research methodology, requirements analysis, and technology selection decisions that guided the development of the HPC Sorting Serious Game.

Chapter 3

Methodology

This chapter describes the research methodology, requirements analysis, design approach, and technology selection decisions that guided the development of the HPC Sorting Serious Game. The methodology follows a Design Science Research approach, emphasizing iterative development and evaluation.

3.1 Research Methodology

3.1.1 Design Science Research Framework

This thesis adopts the Design Science Research (DSR) methodology [6, 7], which is particularly well-suited for developing innovative artifacts that address practical problems while contributing to theoretical knowledge.

DSR Process Model: The DSR process consists of six iterative activities:

1. **Problem Identification and Motivation**: Identify the research problem (teaching HPC concepts) and justify the value of a solution (web-based serious game).
2. **Define Objectives of a Solution**: Infer solution objectives from problem definition and existing knowledge (create engaging, accessible, educational multiplayer game).
3. **Design and Development**: Create the artifact (HPC Sorting Serious Game) by selecting appropriate technologies and implementing game mechanics.
4. **Demonstration**: Demonstrate the artifact's utility by applying it to the problem context (students learning parallel computing).
5. **Evaluation**: Observe and measure how well the artifact supports a solution to the problem (usability testing, performance benchmarking).
6. **Communication**: Communicate the problem, artifact, evaluation, and contributions to relevant audiences (thesis, potential publications).

Rationale for DSR: DSR is appropriate for this research because:

- The goal is to create a functional artifact (serious game) rather than merely analyze existing phenomena
- The problem is practical and relevant to HPC education
- The solution requires integrating knowledge from multiple domains (pedagogy, game design, parallel computing, web development)
- Evaluation can be both utility-based (does it work?) and design-based (is it well-constructed?)

3.2 Requirements Analysis

A comprehensive requirements analysis was conducted to ensure the game met educational and functional objectives.

3.2.1 Educational Requirements

The primary purpose of the game is education. Requirements were derived from HPC learning objectives and parallel computing curricula:

ER1: OpenMP Simulation

- The game must simulate shared-memory parallelism
- All players must see all cards in the main container (shared memory)
- Players must have public buffers (thread-local storage)
- No explicit communication required between players (implicit synchronization)

ER2: Sequential Execution Baseline

- The game must provide a single-player mode for sequential execution comparison
- A single player must sort all cards without parallelism
- This mode serves as a baseline for understanding speedup from parallelization
- Players should be able to compare sequential vs. parallel performance

ER3: Performance Feedback

- The game must provide timing information (simulating speedup measurement)
- Move counting must encourage algorithmic efficiency
- Visual feedback must reinforce correct and incorrect actions

ER4: Progressive Difficulty

- Support variable numbers of cards (10–200) for scalability lessons
- Support variable numbers of players (2–4) for parallelism degree
- Provide different sorting challenges (random, partially sorted, reverse sorted)

ER5: Conceptual Clarity

- Game mechanics must clearly map to HPC concepts
- Visual design must distinguish shared vs. private data
- Terminology should align with parallel computing vocabulary when appropriate

3.2.2 Functional Requirements

Functional requirements specify what the system must do:

FR1: Card Management

- Generate configurable numbers of cards with unique values
- Shuffle cards randomly at game start
- Render cards with clear visual representation (number, state)
- Support dragging and dropping cards between containers

FR2: Single-Player Mode

- Provide a single-player game mode representing sequential execution
- Display all cards in a scrollable container
- Provide work buffer zones for temporary storage
- Validate sorting order upon completion
- Display completion time and move count

FR3: Multiplayer Mode

- Enable 2–4 players to connect via server-relayed networking
- Provide lobby for matchmaking and game configuration
- Display shared container visible to all players (OpenMP shared memory)
- Synchronize card movements across all clients
- Provide private buffers for each player (thread-local storage)
- Hide cards in other players' private buffers (thread-private data)
- Handle player disconnections gracefully

FR4: User Interface

- Main menu with navigation to single-player and multiplayer modes
- Lobby interface for creating/joining games
- In-game UI with timer, move counter, player indicators
- Victory screen with performance summary
- Settings for theme, controls, and game parameters

FR5: Feedback Systems

- Toast notifications for events (player joined, card moved, errors)
- Visual animations for card pickup, drop, invalid moves
- Sound effects for actions (optional, can be disabled)
- Confetti or celebration effects upon game completion

FR6: Quality Attributes

- Target 60 FPS in modern web browsers
- Game initialization and card generation under 2 seconds
- Support up to 200 cards without significant performance degradation
- Intuitive interface requiring no tutorial for basic gameplay
- Responsive drag-and-drop with visual feedback

- Graceful handling of network disconnections
- Clean code structure with modular design
- Open-source license (MIT) for educational reuse
- No account creation or authentication required

3.2.3 Core Concept

The game presents players with a deck of numbered cards that must be sorted in ascending order. This simple metaphor maps directly to parallel sorting algorithms, with different game modes representing different execution paradigms:

Single-Player Mode (Sequential Execution): Represents traditional sequential algorithm execution where a single process sorts all data. The player works alone with all cards visible, performing the entire sorting task independently.

Multiplayer Mode (OpenMP Shared-Memory Parallelism): Represents shared-memory parallelism where multiple threads cooperate on a single address space. All players work collaboratively with cards visible in a shared container (representing shared memory), each player uses private buffer zones for local sorting (representing thread-local storage), and players work simultaneously without forced turn-taking (representing OpenMP's implicit synchronization and parallel execution model).

The complete architecture and pedagogical mapping are detailed in Chapter 4.

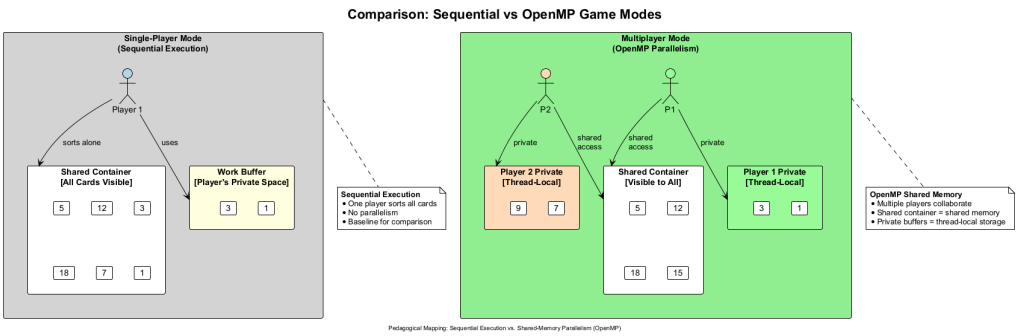

Figure 3.1: Comparison of game modes: Single-player represents sequential execution (left) where one player sorts all cards independently; Multiplayer represents OpenMP shared-memory parallelism (right) where multiple players collaborate with a shared container and private buffers

3.3 Technology Selection

Technology choices were treated as methodological outcomes of iterative experimentation, not fixed assumptions. This section documents the evaluation process, alternatives explored, observed constraints, and final decisions for both core architecture and the supporting development toolchain.

3.3.1 Tool and Framework Evaluation Process

Selection decisions were made using a consistent set of criteria aligned with thesis goals:

- **Educational fit**: ability to support clear mapping between gameplay mechanics and HPC concepts
- **Web deployment feasibility**: practical browser support and export reliability
- **Multiplayer support**: synchronization model maturity and debugging feasibility
- **Ecosystem maturity**: documentation quality, examples, and community responsiveness
- **Debugging observability**: runtime inspection and logging support
- **Development speed**: iteration time and implementation complexity
- **Maintainability**: modularity, replacement cost, and long-term project sustainability

Decisions were validated through prototype iterations described in Section 3.3.8, with problematic assumptions revisited as implementation evidence emerged.

3.3.2 Game Engine: Godot 4.x

Alternatives explored: Unity, Unreal, Godot, and Cocos2d were evaluated against the criteria in Section 3.3.1.

Practical background and evaluation context: Before this thesis, the author had only brief prior hands-on exposure to Unity and Godot, and no production experience with Unreal or Cocos2d. For this reason, engine selection emphasized thesis-time experimentation and iteration evidence rather than long-term prior specialization.

Unity: Unity was a strong candidate due to its mature ecosystem, large community, and broad platform support. It provides robust tooling and many ready-made resources, which can reduce implementation risk. However, for this specific web-first educational prototype, the overall project/runtime footprint and workflow complexity were less aligned with the goal of fast, lightweight iteration.

Unreal Engine: Unreal offers advanced rendering and strong networking capabilities, but it was less suitable for a small 2D educational game with frequent design revisions. In practice, heavier tooling and longer compilation/build cycles can slow down rapid prototyping during thesis development, where many short iteration loops are required.

Cocos2d: Cocos2d is efficient for 2D applications and supports cross-platform development. However, for this thesis it appeared less convenient as an all-in-one workflow for scene editing, multiplayer experimentation, and rapid educational UI iteration. Compared to Godot's integrated editor-first experience, Cocos2d would likely require more custom assembly of tooling and architecture choices for comparable development speed.

Godot: Godot provided the best balance for this thesis context: small web exports, strong 2D workflow, open-source licensing, and fast iteration through a cohesive scene-based architecture. These factors made it the most natural fit for implementing and evaluating the proposed educational game within thesis time constraints.

Table 3.1: Game engine comparison for HPC Sorting Game

Criterion	**Unity**	**Unreal**	**Godot**	**Cocos2d**
Free/Open Source	Partial	Partial	Yes	Yes
2D Performance	Good	Fair	Excellent	Excellent
Web Export	Good	Fair	Excellent	Good
Learning Curve	Medium	Steep	Gentle	Medium
Networking APIs	Good	Excellent	Good	Fair
Community Size	Large	Large	Medium	Small
Export Size	Large	Very Large	Small	Medium

Observed constraints:

- Web target required small export size and predictable browser behavior
- Educational project constraints favored open-source tooling and no licensing risk
- 2D interaction with many UI elements required efficient scene and layout workflows

Decision and justification: Godot Engine was selected because:

1. **Cost**: Completely free and open-source (MIT license), no royalties, subscriptions, or hidden fees. Critical for educational projects with no budget.
2. **2D Optimization**: Godot is specifically optimized for 2D games, unlike Unity and Unreal which prioritize 3D. This results in better performance and smaller export sizes.
3. **Lightweight**: Web export builds are significantly smaller (10–20 MB) compared to Unity (50–100 MB) or Unreal (100+ MB), important for fast loading in browsers.
4. **GDScript**: Python-like scripting language is accessible to students who may examine the source code. Lower barrier to entry than C# (Unity) or C++ (Unreal/Cocos2D).
5. **Scene Architecture**: Godot's scene-based architecture promotes modular design and code reuse, aligning well with software engineering best practices.
6. **Built-in Multiplayer**: High-level networking APIs available, though third-party frameworks (GDSync) and custom transport layers were needed for web browser compatibility.
7. **Active Community**: Growing community with extensive tutorials, documentation, and plugins. Version 4.x introduced significant improvements over 3.x.
8. **Ethical Alignment**: Open-source philosophy aligns with educational values and ensures long-term availability without corporate dependencies.

Trade-offs: Godot's limitations include:

- Smaller community and fewer third-party assets compared to Unity
- Fewer commercial games as reference implementations
- Some plugins less mature than Unity Asset Store equivalents
- 3D capabilities lag behind Unity/Unreal (not relevant for this project)

3.3.3 Programming Language: GDScript

Alternatives explored: Godot supports three primary programming approaches:

1. **GDScript**: Native scripting language, Python-like syntax
2. **C#**: .NET support via Mono runtime
3. **C++**: Native modules via GDNative/GDExtension

Observed constraints:

- Rapid iteration was necessary during repeated gameplay and networking experiments
- Implementation needed to remain readable for thesis communication and educational reuse
- Cross-platform build complexity needed to remain low

Decision and justification: GDScript was chosen because:

- **Integration**: Tight integration with Godot Engine, optimized for game scripting
- **Simplicity**: Python-like syntax is readable and approachable for students
- **Rapid Development**: Dynamic typing and concise syntax accelerate prototyping
- **Documentation**: Most Godot tutorials and examples use GDScript
- **Performance**: Sufficient for 2D game logic; bottlenecks typically in rendering, not scripting
- **No Build Step**: Interpreted language—no compilation required for iteration

Performance Considerations: GDScript is slower than C++ but acceptable for this use case because:

- Card sorting logic is not computationally intensive
- Godot's rendering engine (written in C++) handles performance-critical operations
- Network latency dominates over local computation time
- Development speed more important than raw execution speed

3.3.4 Multiplayer Framework: GDSync

Alternatives explored: Three approaches were considered for multiplayer networking:

1. **Raw Godot Networking**: Use built-in `ENetMultiplayerPeer` and RPC system
2. **GDSync Framework**: Third-party addon providing high-level synchronization abstractions
3. **Custom Protocol**: Build custom WebRTC integration from scratch

Observed constraints:

- Multiplayer state complexity required reusable abstractions, not only low-level RPC wiring
- Browser deployment exposed transport limitations for native WebRTC assumptions
- Debuggability and incremental adoption were critical during issue diagnosis

Decision and justification: GDSync was selected because:

- **Abstraction Level**: Provides higher-level abstractions than raw Godot networking, reducing boilerplate code
- **State Synchronization**: Built-in support for synchronizing node properties automatically
- **RPC Simplification**: Cleaner RPC syntax and management compared to Godot's built-in system
- **Active Development**: Maintained project with community support
- **Documentation**: Adequate documentation and examples for common use cases
- **Compatibility**: Works well with Godot 4.x and WebRTC connections

Challenges Encountered: GDSync introduced some difficulties:

- **Protected Mode Issue**: Default "protected" mode blocked RPC calls between peers, requiring workarounds (see Chapter 6)
- **Documentation Gaps**: Some features underdocumented or unclear
- **Debugging**: Opaque error messages complicated troubleshooting
- **Framework Overhead**: Additional layer of abstraction sometimes obscured underlying networking behavior

Despite these challenges, GDSync provided net benefits by significantly reducing the amount of low-level networking code required. During development, a framework issue relevant to web export behavior required a local fork maintained by the author to apply a fix [8]. A pull request was submitted to the upstream GDSync repository [9], but until that change is merged and released, this project uses the author's fork for the web-export-based game build.

3.3.5 Networking Technology: HTTP/SSE Relay

Alternatives explored:

- Native peer-to-peer approaches through WebRTC
- Raw Godot networking patterns (not browser-compatible as implemented)
- Relay-based architecture with standard web protocols

Observed constraints: Web browsers impose significant restrictions on networking that prevented use of traditional approaches:

- **No UDP Access**: Browsers cannot open raw UDP sockets, eliminating ENet and similar protocols
- **WebRTC Complexity**: While browsers support WebRTC, GDSync's native WebRTC implementation does not function in browser-exported Godot games due to sandboxing
- **No LAN Discovery**: Browser security prevents local network peer discovery

Decision and justification: HTTP + Server-Sent Events (SSE) To overcome browser limitations, a custom relay server architecture was developed:

1. **HTTP POST for Commands**: All game packets sent via standard HTTP POST requests
2. **SSE for Real-time Events**: Server-Sent Events provide push notifications to clients
3. **Server Relay**: All communication passes through the relay server rather than peer-to-peer
4. **Star Topology**: Clients connect to server; host receives, processes, and broadcasts via server

Relay Server Implementation: The relay server (Python/aiohttp) provides:

- `POST /api/lobby/connect` — Client registration and peer ID assignment
- `POST /api/lobby/create` — Create lobby with room code
- `POST /api/lobby/join` — Join existing lobby by code
- `POST /api/lobby/broadcast` — Send game packets to other peers
- `GET /api/lobby/events` — SSE stream for receiving real-time events

Trade-offs: This architecture has trade-offs compared to peer-to-peer approaches:

- **Advantage**: Universal browser compatibility without plugins
- **Advantage**: Simplified connection establishment (no STUN/TURN configuration)
- **Trade-off**: Requires running relay server infrastructure
- **Trade-off**: Higher latency due to server hop (acceptable for turn-based card game)

3.3.6 Supporting Development Toolchain

Beyond core engine/networking choices, several tools were selected to improve debugging observability, delivery speed, and reproducibility.

- **ToastParty (Godot plugin)**: non-blocking UI event popups; improved usability feedback during multiplayer sessions [10].
- **VarTree (Godot plugin)**: runtime inspection/modification of sync variables; reduced debugging turnaround [11].
- **Logger/custom logging client**: structured network/gameplay logs; improved diagnosis of timing and replication issues.
- **Python/aiohttp relay server**: browser-compatible signaling/relay backend for HTTP/SSE transport [12, 13].
- **Ruff**: linting and style checks for Python server/scripts; improved maintainability and consistency [14].
- **justfile**: repeatable task automation for build/test/maintenance workflows [15].
- **mkcert**: trusted local TLS certificates for secure browser multiplayer testing [16].
- **Chocolatey**: standardized Windows package setup, reducing environment variability [17].
- **const_generator**: generated constants to reduce manual duplication and config drift [18].
- **Git + GitHub**: version control, issue tracking, and collaborative documentation supporting reproducible research development [19, 20].

HTTP/SSE Relay - Message Flow

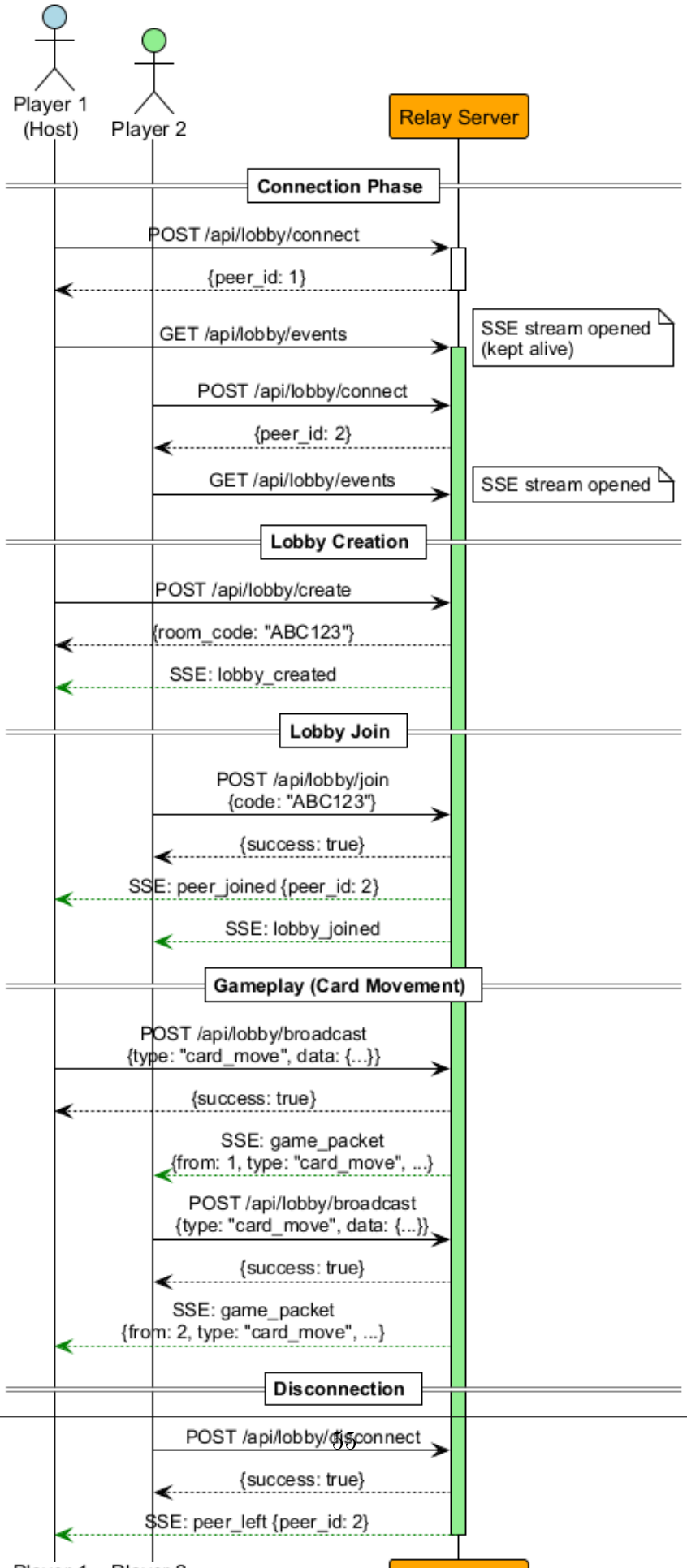

Table 3.2: Final toolchain decisions and accepted trade-offs

Tool	Category	Selected because	Trade-off accepted
Godot 4.x	Engine	Fast 2D/web workflow, open-source, small exports	Smaller ecosystem than Unity/Unreal
GDScript	Language	Fast iteration and readability	Lower peak performance than C++
GDSync	Multiplayer framework	High-level synchronization abstractions	Framework quirks and debugging overhead
HTTP/SSE relay (Python)	Transport	Browser compatibility and implementation control	Extra relay infrastructure and hop latency
Supporting plugins/utilities	Toolchain	Improved debugging, quality, and reproducibility	Additional maintenance surface

3.3.7 Final Toolchain and Rationale Summary

3.3.8 Development Approach

The game development followed an iterative, incremental approach:

Phase 1: Concept and Prototyping (Weeks 1–3)

- Defined core mechanics based on physical card-sorting experiments
- Created paper prototypes and mockups
- Evaluated technology options (game engines, frameworks)
- Developed minimal prototype with basic card rendering

Phase 2: Single-Player Implementation (Weeks 4–7)

- Implemented card generation, shuffling, and rendering
- Developed drag-and-drop mechanics for touch and mouse input
- Created buffer zones for private card storage
- Implemented sorting validation and game completion logic
- Added timer and move counting

Phase 3: Multiplayer Foundation (Weeks 8–12)

- Developed HTTP/SSE relay server for web browser compatibility
- Implemented lobby system for matchmaking
- Developed basic state synchronization
- Integrated GDSync framework with custom web transport layer
- Tested connection establishment and stability

Phase 4: Multiplayer Refinement (Weeks 13–16)

- Implemented card visibility management (private buffers)
- Developed host-authoritative state management
- Added disconnection handling and recovery
- Refined synchronization for card movements
- Addressed framework issues (GDSync protected mode)

Phase 5: Polish and Testing (Weeks 17–20)

- UI/UX improvements for web usability
- Performance optimization (rendering, networking)
- Extensive testing across web browsers
- Documentation and code cleanup
- Preparation for evaluation and thesis writing

3.4 Game Design Methodology

3.4.1 Mapping HPC Concepts to Game Mechanics

The core challenge was translating abstract parallel computing concepts into tangible game mechanics. Table 3.3 shows the systematic mapping process:

Table 3.3: Systematic mapping of HPC concepts to game mechanics

HPC Concept	**Real-World Analogy**	**Game Mechanic**
Sequential Execution	Single student working alone	Single player in solo mode
Thread (OpenMP)	Student with access to shared desk	Player in multiplayer mode
Shared Memory	Desk visible to all students	Shared card container
Thread-Local Storage	Student's private notebook	Player's private buffer zones
Parallel Execution	Students work simultaneously	Multiple players, no turn-taking
Synchronization	Students coordinate access	Shared container access patterns
Memory Access Overhead	Time to reach shared desk	Network latency to shared data
Speedup	Multiple students finish faster	Timer comparison: solo vs. multi
Load Balancing	Even distribution of work	Fair work distribution among players

Design Validation: The pedagogical mapping was validated through:

- Comparison with Professor D'Agostino's physical experiments
- Review by HPC instructors for conceptual accuracy
- Pilot testing with students familiar with parallel computing
- Iterative refinement based on feedback

3.4.2 User Experience Design

Design Principles:

1. **Minimize Cognitive Load**: Simple, clear visuals; avoid information overload
2. **Immediate Feedback**: Every action produces visual/audio response
3. **Progressive Disclosure**: Show information when needed, hide complexity initially
4. **Error Prevention**: Design interface to prevent common mistakes
5. **Aesthetic Simplicity**: Clean, uncluttered design focuses attention on cards

Interaction Design: Card interactions were designed to work with both touch and mouse input:

- **Click/Tap to Select**: Quick click or tap selects a card (alternative to drag)
- **Drag to Move**: Click-and-hold initiates drag, release drops card
- **Scroll**: Vertical scroll moves through card container
- **Visual Feedback**: Cards scale up when dragged, drop zones highlight

Color Coding: Colors convey semantic meaning:

- **Blue**: Normal cards, main container
- **Yellow**: Highlighted cards during drag
- **Gray**: Disabled/hidden cards (other players' buffers)
- **Purple**: Player-specific UI elements (buffers, indicators)

3.5 Evaluation Methodology

Evaluation Criteria: The educational tool's success was evaluated across multiple dimensions:

1. **Technical Performance**: Frame rate, memory usage, browser compatibility
2. **Functional Completeness**: Implementation of all required features
3. **Usability**: Ease of use, learnability, satisfaction
4. **Educational Effectiveness**: Preliminary conceptual understanding and engagement indicators (informal evaluation scope)

Evaluation Methods:

3.5.0.0.0.1 Technical Benchmarking:

- Performance profiling using Godot's built-in tools
- Frame rate monitoring in target browsers
- Memory usage tracking during extended gameplay

3.5.0.0.0.2 Usability Testing:

- Think-aloud protocols with test users
- Observation of first-time players
- Task completion rates and times
- System Usability Scale (SUS) questionnaire (if time permits)

3.5.0.0.0.3 Educational Assessment:

- Pre/post-test knowledge assessments (if formal study conducted)
- Concept mapping to evaluate mental model formation
- Comparison with traditional teaching methods (if feasible)
- Qualitative feedback on perceived learning value

3.6 Development Tools and Environment

Development Platform:

- **OS**: Windows 11 / Linux (Ubuntu 22.04)
- **IDE**: Visual Studio Code with GDScript extension
- **Engine**: Godot Engine 4.5.x
- **Version Control**: Git with GitHub repository

Testing Environment:

- Primary: Chrome 120+ (Windows, macOS)
- Secondary: Firefox 120+ (Windows, Linux)
- Tertiary: Safari 17+ (macOS)
- Local development server for testing

Build and Deployment:

- Web export (HTML5) built via Godot export templates
- Debug builds for development testing
- Release builds with optimizations for distribution
- Static file hosting for deployment

3.7 Summary

This chapter described the comprehensive methodology employed to develop the HPC Sorting Serious Game:

- **Research Approach**: Design Science Research framework with iterative development
- **Requirements**: Educational and functional requirements derived from learning objectives
- **Technology Selection**: Justified choices of Godot Engine, GDScript, GDSync, and HTTP/SSE relay architecture based on project needs

- **Game Design**: Systematic mapping of HPC concepts to game mechanics with UX design principles
- **Evaluation**: Multi-faceted evaluation approach combining technical, usability, and educational assessment

The next chapter presents the detailed system architecture resulting from these methodological decisions, including scene structure, component design, and multiplayer networking architecture.

Chapter 4

System Design and Architecture

This chapter presents the system architecture decisions that enabled the HPC Sorting Serious Game to be feasible, maintainable, and pedagogically interpretable. It focuses on architectural structure and rationale rather than exhaustive implementation listings. Detailed implementation is provided in Chapter 5, and educational/-operational effectiveness is evaluated in Chapter 7.

4.1 Architectural Overview

The architecture is organized to support two complementary goals:

- **Pedagogical clarity**: game mechanics must remain interpretable as OpenMP-inspired shared-memory collaboration.
- **Technical robustness**: multiplayer synchronization must remain consistent across browser clients under variable latency.

4.1.1 Design Principles

The architecture is guided by the following principles:

1. **Component modularity**: cards, buffers, and managers are separated so mechanics can evolve without rewriting the whole system.
2. **Separation of concerns**: UI, game rules, and synchronization logic are isolated to reduce coupling during iterative experiments.
3. **Host-authoritative conflict handling**: one authority resolves contested actions to prevent divergent multiplayer states.
4. **Observable synchronization**: architecture includes explicit state transitions that can be logged and inspected during debugging.
5. **Web-first constraints**: transport, update cadence, and payload design prioritize browser compatibility over engine-native networking assumptions.

4.1.2 System Layers

Presentation Layer Renders cards, buffers, indicators, and interaction feedback.

Game Logic Layer Validates sorting actions, completion conditions, and per-mode rules.

Synchronization Layer Coordinates shared/private state propagation, ordering, and conflict resolution.

Transport/Relay Layer Handles browser-compatible signaling and event relay via HTTP/SSE.

These layers decouple pedagogical behavior from transport mechanics, which was essential when adapting multiplayer behavior for browser export.

4.2 Scene and Interaction Architecture

The scene model is intentionally concise: a menu/lobby entry path and two gameplay contexts (single-player baseline and multiplayer OpenMP-style collaboration). The scene split supports direct comparison between sequential and collaborative execution while reusing core card and validation logic.

Architectural Intent:

- Single-player provides the sequential baseline for speedup interpretation.
- Multiplayer preserves shared container semantics while enforcing private buffer visibility.
- Lobby separates session orchestration from gameplay state transitions.

This scene structure keeps learning signals (who can see what, who can move what, when state is final) explicit for both users and evaluators.

4.2.1 Scene Lifecycle and Transition Logic

Scene changes are managed through a central scene manager to avoid ad-hoc transitions across gameplay scripts. This keeps navigation behavior deterministic and makes it easier to relate transitions to teaching intent (e.g., sequential baseline first, then collaborative mode).

4.2.2 Event-Driven Coordination

The architecture is event-driven at component boundaries. Local game objects communicate via Godot signals, while multiplayer state changes are propagated through authority-managed synchronization paths. This split allows UI responsiveness without directly coupling UI events to transport internals.

4.3 Core Component Roles

Rather than duplicating full class implementations, this section summarizes architecture-critical component responsibilities.

Card Component (Interaction Unit):

- Encapsulates value identity, interaction state, and visual feedback states.
- Emits events consumed by manager-level logic rather than mutating global state directly.

Card Manager (Rule and State Coordinator):

- Owns sorting validity checks and authoritative placement updates.
- Serves as the integration boundary between UI interaction and synchronization rules.
- Was designed for reusability: shared logic lives in `card_manager.gd` so mode-specific behavior can be layered instead of duplicated.

Multiplayer Card Manager (Authority Extension):

- Explicitly extends `card_manager.gd` as `multiplayer_card_manager.gd`, adding host-authoritative synchronization and reconciliation behavior while reusing baseline card/rule logic.
- Enforces shared/private visibility semantics across peers.

4.4 Multiplayer Architecture

4.4.1 Network Topology

The implemented multiplayer topology uses browser-compatible relay patterns for session events and state updates. This avoids assumptions that are unstable in

browser-exported runtime contexts.

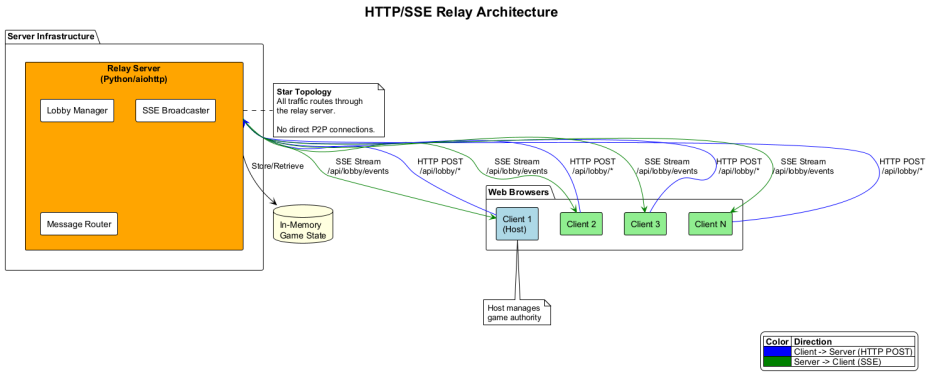

Figure 4.1: HTTP/SSE relay-oriented architecture used for web-first multiplayer synchronization

4.4.2 State Synchronization Patterns

Two synchronization patterns are central:

- **Optimistic local interaction**: local responsiveness is preserved while awaiting authoritative confirmation.
- **Host broadcast reconciliation**: final shared state is re-broadcast to all peers to converge state.

```
# Client proposes move -> host validates -> host broadcasts
    canonical state
on_client_move(card_id, target_slot):
    if host_validate(card_id, target_slot):
        host_apply(card_id, target_slot)
        broadcast_canonical_state(card_id, target_slot)
    else:
        reject_and_restore(card_id)
```

Listing 4.1: Architectural pattern: host-authoritative move confirmation

This pattern minimizes permanent divergence while preserving interaction fluidity.

4.4.3 GDSync Integration Boundary

GDSync is treated as a synchronization abstraction layer, while architecture ownership remains in the game managers. This prevented framework-specific behavior from leaking into all gameplay components and reduced replacement cost when debugging framework limitations. The web-export signaling patch details are intentionally analyzed in Chapter 6, Section 6.3.3, while this section keeps the architectural boundary view.

```
@GDSync.rpc(call_local=true)
func sync_move(card_id: int, target_id: int):
    multiplayer_manager.apply_authoritative_move(card_id, target_id
    )
```

Listing 4.2: Architectural boundary: sync wrapper delegates to game manager

4.4.4 Connection Management

Connection handling is split into phases:

- **Lobby phase**: peer discovery, readiness, and session configuration.
- **Gameplay phase**: synchronized interaction and reconciliation.
- **Failure phase**: disconnection detection and graceful degradation.

4.5 Data Flow and Visibility Model

The architecture explicitly distinguishes:

- **Shared-visible state**: container cards and game-level counters.
- **Player-private state**: cards in private buffers hidden from other players.

This model is central to the OpenMP analogy: all participants can access shared memory, while local buffer state remains private.

```
if card.in_private_buffer():
    card.visible = (card.buffer_owner_id == local_player_id)
else:
    card.visible = true
```

Listing 4.3: Architectural mechanism: private buffer visibility gate

4.6 Barrier Synchronization Architecture

The game includes a barrier synchronization feature that directly models the `#pragma omp barrier` construct from OpenMP. Architecturally, the barrier is implemented as a separate `BarrierManager` component (not embedded in the card manager) following the single-responsibility principle.

4.6.1 State Machine Design

The barrier operates as a three-state finite automaton with the following transitions:

1. **RUNNING** → **WAITING_AT_BARRIER**: Any player initiates a barrier; all must arrive.
2. **WAITING_AT_BARRIER** → **BARRIER_ACTIVE**: All players have reached the barrier; the main thread is granted exclusive access to the shared container.
3. **BARRIER_ACTIVE** → **RUNNING**: The main thread releases the barrier; normal play resumes.

This maps directly to OpenMP barrier semantics: threads reaching the barrier must wait until all threads have arrived, after which execution proceeds in a coordinated fashion.

4.6.2 Integration Points

The `BarrierManager` integrates with the multiplayer card manager through three mechanisms:

- **Signals**: emits `barrier_state_changed` to trigger UI overlays (lock screen for non-main-thread players).
- **GDSync exposure**: barrier functions are registered for remote invocation alongside card synchronization functions.
- **Settings**: BarrierManager reads the barrier mode (first-to-reach or round-robin) from the global `Settings` autoload.

The barrier implementation details are covered in Chapter 5, Section 5.5.

4.7 Responsive UI/UX Architecture

UI architecture prioritizes interpretable interaction under constrained viewports:

- Scrollable container strategy for large card sets.
- Buffer-first layout to reinforce shared/private distinction.
- Immediate feedback (toast/status indicators) to reduce action ambiguity in multiplayer.

4.7.1 UI Layering and Overlay Isolation

Overlay and inspection elements are separated from gameplay nodes to minimize accidental coupling between debug/auxiliary UI and gameplay transforms. This layering approach supports live inspection tooling while preserving gameplay interaction consistency.

A playful evasive hover animation is also present in the main menu as a non-critical engagement microinteraction; its reception in early tests is discussed in Chapter 7, Section 7.1.5.

4.8 Performance and Scalability Constraints

Architecture-level performance strategies include:

- bounded synchronization payloads,
- authoritative reconciliation instead of full scene replication,
- UI/container policies that remain usable at higher card counts.

Observed constraints and measured outcomes are discussed in Chapter 7.

4.9 Summary

This chapter documented architectural decisions that made the tool technically feasible and pedagogically interpretable in a web-first multiplayer context. The focus was on synchronization boundaries, state visibility semantics, and authority patterns rather than exhaustive implementation detail. Detailed implementation evidence is provided in Chapter 5, while educational and operational effectiveness is evaluated in Chapter 7.

Chapter 5

Implementation

This chapter presents key implementation mechanisms of the HPC Sorting Serious Game. It covers the core game logic, multiplayer synchronization protocol, barrier synchronization (a direct pedagogical mapping to `#pragma omp barrier`), the web-export compatibility patch, and development observability tooling. Tool/framework selection rationale is documented in Chapter 3, architectural decisions in Chapter 4, and effectiveness evaluation in Chapter 7.

The complete source code is available in the project repository [21]. External tool and framework links are listed in Appendix D.9.

5.1 Implementation Focus Areas

The implementation evidence is organized around six high-impact concerns:

- card management and sorting correctness,
- component reuse through inheritance (`CardManager` → `MultiplayerCardManager`),
- network state serialization and synchronization protocol,
- barrier synchronization as a pedagogical feature,
- web-export compatibility patching,
- observability and debuggability for multiplayer faults.

5.2 Card Management and Sorting Correctness

The single-player card management logic resides in `card_manager.gd` (705 lines), which serves as both the standalone game controller and the base class for multiplayer extension.

5.2.1 Card Generation and Initialization

Card generation supports configurable counts (10–200) with optional unique or repeated values. The initialization sequence in `_ready()` follows a deterministic pipeline:

```
func _ready():
    if num_cards < 1:
        num_cards = 1
    values = generate_random_values()
    sorted_all = values.duplicate()
    sorted_all.sort()

    adjust_container_spacing()
    cards_array = generate_completed_card_array(values)
    sorted_cards_array = generate_completed_card_array(sorted_all,
   "SortedCard_")
    fill_card_container(cards_array, card_container)
    fill_card_container(sorted_cards_array, sorted_cards_container)
    slots = create_buffer_slots()
    _connect_signals()
```

Listing 5.1: Card manager initialization pipeline (card_manager.gd)

Each card is instantiated from a preloaded scene, assigned a numeric value and a color derived from the value (for visual distinctiveness), and placed in a scrollable `HBoxContainer`. A parallel sorted reference array is generated so players can optionally view the target ordering.

5.2.2 Sorting Validation

Sorting validation is the completion contract used in both single-player and multiplayer modes. The actual implementation iterates over the card container's children (the authoritative UI ordering):

```
func check_sorting_order() -> bool:
    var sorted_correctly = true
    if card_container.get_child_count() != num_cards:
        return false
    var cards_in_container: Array[Card] = []
    for child in card_container.get_children():
        if child is Card:
            cards_in_container.append(child as Card)
    for i in range(1, cards_in_container.size()):
        var current_card = cards_in_container[i].value
        var previous_card = cards_in_container[i - 1].value
        if current_card < previous_card:
            sorted_correctly = false
            break
    return sorted_correctly
```

Listing 5.2: Sorting validation (card_manager.gd, line 645)

This mechanism provides a deterministic completion criterion independent of UI animation state and network timing. Notably, validation checks the *scene tree child order* rather than an abstract array, ensuring that what the player sees matches what is validated (see Chapter 7, Section 7.6).

5.2.3 Buffer Contiguity Check

Players use private buffer zones (simulating thread-local storage) to temporarily hold cards during sorting. A buffer contiguity check validates that cards in the buffer form a contiguous subsequence of the sorted target:

```
func check_player_buffer_contiguity() -> bool:
    var buffer_values = []
    for slot in slots:
        if slot.occupied_by == null:
            return false
        buffer_values.append(slot.occupied_by.value)
    return SubarrayUtils.is_contiguous_subarray(sorted_all,
   buffer_values)
```

Listing 5.3: Buffer contiguity validation (card_manager.gd, line 693)

This constraint reinforces the pedagogical mapping: just as a thread in an OpenMP program should work on a logically coherent subset of data, players must select contiguous card ranges for their buffers.

5.3 Component Reuse: CardManager Inheritance

The multiplayer game controller, `MultiplayerCardManager` (1143 lines), explicitly extends the single-player `CardManager`:

```
extends "res://scenes/CardScene/scripts/card_manager.gd"
class_name MultiplayerCardManager
```

Listing 5.4: Multiplayer extension declaration (multiplayer_card_manager.gd, line 1)

This inheritance structure allows the multiplayer mode to reuse all card generation, UI layout, sorting validation, and buffer management logic while adding:

- host/client role differentiation,
- network state serialization (via `CardState`),
- GDSync function exposure for remote procedure calls,
- barrier synchronization integration.

The multiplayer `_ready()` overrides the parent to branch on host vs. client role:

```
func _ready():
    is_host = ConnectionManager.am_i_host()
    my_client_id = ConnectionManager.get_my_client_id()

    barrier_manager = BarrierManager.new()
    barrier_manager.set_barrier_mode(Settings.barrier_mode)
    barrier_manager.barrier_state_changed.connect(
   _on_barrier_state_changed)

    setup_multiplayer_sync()

    if is_host:
        super._ready()
        await get_tree().process_frame
        broadcast_initial_game_state()
        game_state_synced = true
    else:
        _initialize_client_structure()
        await get_tree().create_timer(0.5).timeout
        request_game_state_from_host()
```

Listing 5.5: Multiplayer initialization with host/client branching (multiplayer_card_manager.gd)

The host executes the parent `_ready()` (generating cards normally) and then broadcasts the initial state. Clients initialize an empty UI structure and request the authoritative game state from the host, avoiding duplicate card generation that would cause divergence.

5.4 Multiplayer Synchronization Protocol

5.4.1 CardState Serialization

All card state transmitted over the network is serialized through a dedicated `CardState` inner class:

```
class CardState:
    var value: int
    var index: int
    var original_index: int
    var in_container: bool
    var in_buffer: bool
    var buffer_owner: int

    func to_dict() -> Dictionary:
        return {
            "value": value,
            "index": index,
            "original_index": original_index,
            "in_container": in_container,
            "in_buffer": in_buffer,
            "buffer_owner": buffer_owner
```

```
        }

    static func from_dict(data: Dictionary) -> CardState:
        return CardState.new(
            data.get("value", 0),
            data.get("index", -1),
            data.get("original_index", -1),
            data.get("in_container", true),
            data.get("in_buffer", false),
            data.get("buffer_owner", -1)
        )
```

Listing 5.6: CardState transport protocol (multiplayer_card_manager.gd, lines 6–49)

`CardState` captures whether a card is in the shared container or in a player's private buffer, and who owns it. This explicit serialization boundary ensures that network payloads are minimal and self-describing, and that deserialization on the receiving client produces a deterministic scene state.

5.4.2 Lobby and Session Lifecycle

The lobby manages session creation/joining and readiness transitions before gameplay authority is established.

1. Player creates room or enters room code
2. Connection to signaling server via HTTP/SSE
3. Peer handshake with host
4. Players appear in lobby list
5. Host configures game settings (card count, buffer size, barrier mode)
6. Host starts game, triggering scene transition to multiplayer gameplay

5.4.3 GDSync Function Exposure

GDSync provides high-level networking abstractions. For this thesis, the project uses an author-maintained GDSync fork that includes a fix required for the web-export workflow [8]. A pull request with this fix was submitted upstream [9].

Rather than using decorator-based annotations, the actual GDSync integration uses explicit function exposure. The `setup_multiplayer_sync()` method registers all remotely callable functions:

```
func setup_multiplayer_sync():
    GDSync.expose_func(self.sync_complete_game_state)
    GDSync.expose_func(self.sync_card_moved)
```

```
    GDSync.expose_func(self.sync_card_entered_buffer)
    GDSync.expose_func(self.sync_card_left_buffer)
    GDSync.expose_func(self.sync_timer_state)
    GDSync.expose_func(self.sync_game_finished)
    GDSync.expose_func(self.send_current_state_to)
    # Barrier synchronization functions
    GDSync.expose_func(self.barrier_thread_reached)
    GDSync.expose_func(self.barrier_activate)
    GDSync.expose_func(self.barrier_card_picked)
    GDSync.expose_func(self.barrier_release)
```

Listing 5.7: GDSync function exposure (multiplayer_card_manager.gd)

Remote calls are then dispatched via `GDSync.call_func()` (broadcast) or `GDSync.call_func_on()` (targeted to a specific peer). This explicit exposure model was required because GDSync's "protected mode" silently drops calls to unexposed functions (see Chapter 6, Section 6.3.1).

5.4.4 State Broadcast and Reconciliation

When a card is placed in the main container (shared memory), the multiplayer manager overrides the parent handler to broadcast the move:

```
func _on_card_placed_in_container(
    dropped_card: Card = null,
    was_in_buffer: bool = false,
    original_slot: Variant = null
):
    super._on_card_placed_in_container()
    await get_tree().process_frame
    var new_index = moved_card.get_index()

    if was_in_buffer:
        GDSync.call_func(
            self.sync_card_left_buffer,
            [moved_card.value, my_client_id, new_index]
        )
    else:
        GDSync.call_func(
            self.sync_card_moved,
            [moved_card.value, moved_card.original_index,
             new_index, my_client_id]
        )
```

Listing 5.8: Card movement broadcast (multiplayer_card_manager.gd)

Clients receiving a `sync_card_moved` call skip the update if it originated from themselves (preventing echo loops), then reposition the card in their local scene tree:

```
func sync_card_moved(
    card_value: int, from_index: int, to_index: int,
   moving_client_id: int
):
    if moving_client_id == my_client_id:
```

```
        return
    var card: Card = _find_card_by_value(card_value)
    if not card:
        push_error("Card not found: ", card_value)
        return
    if card.get_parent() != card_container:
        if card.get_parent():
            card.get_parent().remove_child(card)
        card_container.add_child(card)
    card_container.move_child(card, to_index)
    card.visible = true
    card.set_can_drag(true)
```

Listing 5.9: Remote card movement handler (multiplayer_card_manager.gd)

5.4.5 Visibility Management

In multiplayer mode (OpenMP simulation), players should not see cards in other players' private buffers, representing thread-private data. When a card enters another player's buffer, it is tracked locally and removed from the container:

```
func sync_card_entered_buffer(card_value: int, entering_player_id:
    int):
    if entering_player_id == my_client_id:
        return
    var card: Card = _find_card_by_value(card_value)
    if not card:
        return
    if card.get_parent() == card_container:
        card_container.remove_child(card)
    cards_in_other_buffers[card_value] = entering_player_id
```

Listing 5.10: Buffer entry notification (multiplayer_card_manager.gd)

Cards in `cards_in_other_buffers` are invisible to the local player. When a card leaves another player's buffer, the reverse operation adds it back to the container. This mechanism preserves the shared-vs-private memory distinction that users report recognizing in evaluation sessions (Chapter 7, Sections 7.1.2 and 7.1.4).

5.5 Barrier Synchronization

The barrier synchronization feature is one of the most direct pedagogical mappings in the game: it implements `#pragma omp barrier` semantics in multiplayer gameplay. When a player initiates a barrier, all players must reach the barrier point before a designated "main thread" (one player) can perform coordinated operations on the shared container.

5.5.1 BarrierManager State Machine

The `BarrierManager` class (113 lines) implements a three-state finite automaton:

```
class_name BarrierManager
extends RefCounted

enum BarrierState { RUNNING, WAITING_AT_BARRIER, BARRIER_ACTIVE }

signal barrier_state_changed(new_state: BarrierState)
signal thread_reached_barrier(player_id: int)
signal main_thread_assigned(player_id: int)

var current_state: BarrierState = BarrierState.RUNNING
var threads_at_barrier: Dictionary = {}
var main_thread_id: int = -1
var barrier_mode: int = 0  # 0 = first to reach, 1 = round-robin
```

Listing 5.11: BarrierManager state machine (barrier_manager.gd)

The three states map directly to HPC barrier semantics:

RUNNING All threads (players) work independently—normal gameplay.

WAITING_AT_BARRIER At least one thread has reached the barrier; others must arrive before proceeding.

BARRIER_ACTIVE All threads are at the barrier; the designated main thread can perform coordinated operations while others are blocked.

5.5.2 Main Thread Selection

The barrier supports two modes for selecting which player becomes the "main thread" (coordinator):

```
func determine_main_thread(first_thread_id: int, all_player_ids:
    Array) -> int:
    if barrier_mode == 0:  # First to reach barrier
        return first_thread_id
    else:  # Round-robin
        return _get_next_in_rotation(all_player_ids)

func _get_next_in_rotation(player_ids: Array) -> int:
    var sorted_ids = player_ids.duplicate()
    sorted_ids.sort()
    if last_main_thread_id == -1:
        return sorted_ids[0]
    var last_index = sorted_ids.find(last_main_thread_id)
    var next_index = (last_index + 1) % sorted_ids.size()
    return sorted_ids[next_index]
```

Listing 5.12: Main thread determination (barrier_manager.gd)

The round-robin mode sorts player IDs for consistent ordering across all clients, then rotates through them. This teaches students that coordinator selection in parallel systems requires deterministic agreement between all participants.

5.5.3 Barrier State Transitions

Transitions are enforced through guard conditions:

```
func enter_waiting_state(initiator_id: int, all_player_ids: Array):
    if current_state != BarrierState.RUNNING:
        return
    current_state = BarrierState.WAITING_AT_BARRIER
    main_thread_id = determine_main_thread(initiator_id,
   all_player_ids)
    mark_thread_at_barrier(initiator_id)
    barrier_state_changed.emit(current_state)

func activate_barrier():
    if current_state != BarrierState.WAITING_AT_BARRIER:
        return
    current_state = BarrierState.BARRIER_ACTIVE
    barrier_state_changed.emit(current_state)

func release_barrier():
    last_main_thread_id = main_thread_id
    reset()
```

Listing 5.13: Barrier state transitions (barrier_manager.gd)

When the barrier is active, non-main-thread players see a lock overlay (`BarrierLockOverlay`) that prevents interaction, while the main thread can freely manipulate the shared container. This directly mirrors the semantics of a critical section protected by a barrier: only one thread executes while others wait.

5.6 Web-Export Compatibility Patch

Browser-based deployment required replacing GDSync's native `LocalServer` (which relies on UDP/WebRTC) with an HTTP/SSE-based relay. This section documents the patching mechanism; the problem analysis is in Chapter 6, Section 6.3.3.

5.6.1 Patch Application

The patch is applied at runtime by an autoload script that runs after GDSync initializes:

```
extends Node

var _patch_applied: bool = false

```

```
5 const web_local_server_script = preload(
6     ProjectFiles.Scripts.LOCAL_SERVER_SIGNALING
7 )
8
9 func _ready() -> void:
10     if not OS.has_feature("web"):
11         return
12     _apply_web_patch()
13
14 func _apply_web_patch() -> void:
15     var gdsync = get_node_or_null("/root/GDSync")
16     if not gdsync:
17         logger.log_error("GDSync autoload not found!")
18         return
19
20     var original_local_server = gdsync.get_node_or_null("
    LocalServer")
21     if not original_local_server:
22         logger.log_error("GDSync LocalServer not found!")
23         return
24
25     # Disable original LocalServer
26     original_local_server.name = "LocalServer_Original_Disabled"
27     original_local_server.set_process(false)
28     original_local_server.set_physics_process(false)
29
30     # Replace with web-compatible LocalServer
31     var web_local_server = web_local_server_script.new()
32     web_local_server.name = "LocalServer"
33     gdsync.add_child(web_local_server)
34     gdsync._local_server = web_local_server
35
36     if (gdsync._connection_controller
37         and "local_server" in gdsync._connection_controller):
38         gdsync._connection_controller.local_server =
    web_local_server
39
40     _patch_applied = true
```

Listing 5.14: GDSync web patch (gdsync_web_patch.gd)

The patch preserves the existing `LocalServer` API contract: all higher-level GDSync calls (`lobby_create`, `call_func`, etc.) continue to work without modification. Only the transport layer is replaced, validating the architectural boundary described in Chapter 4, Section 4.4.3.

5.6.2 Connection Manager Signal Architecture

The `ConnectionManager` autoload bridges GDSync framework signals to game-level events through a centralized wiring pattern:

```
1 func _ready():
2     GDSync.connected.connect(_on_gdsync_connected)
3     GDSync.connection_failed.connect(_on_gdsync_connection_failed)
```

```
    GDSync.lobby_created.connect(_on_gdsync_lobby_created)
    GDSync.lobby_creation_failed.connect(
   _on_gdsync_lobby_creation_failed)
    GDSync.client_joined.connect(_on_gdsync_client_joined)
    GDSync.client_left.connect(_on_gdsync_client_left)
    GDSync.lobby_joined.connect(_on_gdsync_lobby_joined)
    GDSync.lobby_join_failed.connect(_on_gdsync_lobby_join_failed)
    GDSync.lobbies_received.connect(_on_gdsync_lobby_list_updated)

    GDSync.expose_func(ToastParty.show)
    GDSync.expose_var(self, "actual_lobby_host_id")
```

Listing 5.15: GDSync signal wiring (connection_manager.gd)

This centralized approach ensures that all GDSync events are handled in one place with consistent error mapping and logging, rather than being scattered across gameplay scripts. Internally, `ConnectionManager` maintains a typed player map (`MultiplayerTypes.Pl` and emits its own higher-level signals (e.g., `player_joined_lobby`, `player_list_updated`) that lobby and gameplay scenes subscribe to.

5.7 Debugging and Development Observability

5.7.1 Structured Logger Integration

A custom `ColorfulLogger` class provides per-component categorized logging. Each script obtains its own logger instance via:

```
@onready var logger := CustomLogger.get_logger(self)
```

Listing 5.16: Logger instantiation pattern

This pattern tags every log line with the emitting node's name and supports `log_info`, `log_warning`, `log_error`, and `log_debug` levels. During multiplayer debugging, these structured logs were essential for correlating events across multiple browser instances, exposing timing/order failures that were otherwise hard to reproduce (Chapter 6, Section 6.3.4).

5.7.2 Runtime Variable Inspection

The VarTree plugin [11] allows real-time inspection and modification of variables during gameplay. The card manager mounts debug variables that update every frame:

```
func _setup_var_tree(vt: VarTree) -> void:
    vt.mount_var(self, "Client number", {
        "font_color": Color.CYAN,
        "format_callback":
            func(_value): return str(Constants.get_game_debug_id())
    })
```

```
    if Settings.is_multiplayer:
        vt.mount_var(self, "dbg_game_mp/IAmHost", {
            "format_callback":
                func(_value): return str(ConnectionManager.
am_i_host())
        })
        vt.mount_var(self, "dbg_game_mp/Players count", {
            "format_callback":
                func(_value): return str(
                    ConnectionManager.get_player_list().size())
        })
```

Listing 5.17: VarTree debug variable mounting (card_manager.gd)

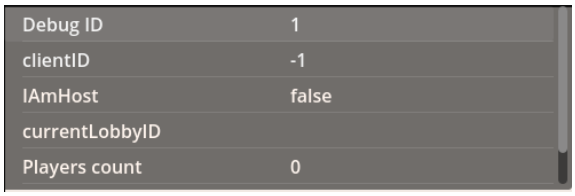

Figure 5.1: VarTree debug panel showing GDSync runtime variables: Debug ID, clientID, IAmHost flag, currentLobbyID, and Players count

This live inspection was particularly valuable during multiplayer debugging sessions where breakpoints were impractical due to timing-sensitive synchronization paths.

5.8 Summary

This chapter presented the core implementation mechanisms of the HPC Sorting Serious Game:

- **Card management** (705 lines in `card_manager.gd`): generation, layout, sorting validation, and buffer contiguity checking.
- **Multiplayer extension** (1143 lines in `multiplayer_card_manager.gd`): host/-client branching, `CardState` serialization protocol, GDSync function exposure, and state broadcast/reconciliation.
- **Barrier synchronization** (113 lines in `barrier_manager.gd`): three-state machine implementing `#pragma omp barrier` semantics with main-thread selection.
- **Web-export patch** (67 lines in `gdsync_web_patch.gd`): runtime LocalServer replacement preserving API contract.
- **Connection management** (348 lines in `connection_manager.gd`): centralized GDSync signal wiring and typed player state.
- **Observability tooling**: structured logging and live variable inspection for multiplayer fault diagnosis.

The key repository entry points for readers who wish to inspect the full implementation:

Table 5.1: Key source file locations

Component	**Path**
Card Manager (base)	`scenes/CardScene/scripts/card_manager.gd`
Multiplayer Card Manager	`scenes/Multiplayer/MultiplayerGame/multiplayer_card_ma`
Barrier Manager	`scenes/Multiplayer/MultiplayerGame/barrier_manager.gd`
Connection Manager	`scenes/Multiplayer/connection_manager.gd`
Multiplayer Lobby	`scenes/Multiplayer/Lobby/multiplayer_lobby.gd`
GDSync Web Patch	`scenes/Multiplayer/GDSyncWebPatch/gdsync_web_patch.gd`
Signaling LocalServer	`scenes/Multiplayer/GDSyncWebPatch/local_server_signali`
Signaling Client	`scenes/Multiplayer/GDSyncWebPatch/signaling_client.gd`
Scene Manager	`scene_manager.gd`
Signaling Server (Python)	`signaling-server/server.py`

Educational and operational effects of these mechanisms are interpreted in Chapter 7, while implementation failure modes and mitigations are documented in Chapter 6.

Chapter 6

Problems and Challenges

This chapter provides an honest and comprehensive analysis of the difficulties encountered during the development of the HPC Sorting Serious Game. Understanding these challenges and their solutions is valuable for future developers of educational multiplayer games and contributes to the body of knowledge in serious game development.

6.1 Overview

Development of a web-first serious game with real-time multiplayer exposed challenges that were tightly coupled: early synchronization faults reduced gameplay trust, UI constraints affected interaction quality, and debugging complexity slowed iteration speed.

The problem space spans multiple domains:

- **Technology selection and framework limitations**: constrained viable implementation paths and introduced replacement/mitigation overhead.
- **Multiplayer state synchronization complexity**: caused divergence risks, ordering issues, and authority conflicts.
- **UI/UX constraints**: touch interaction precision, screen space management, and browser performance variability during collaborative play.
- **Performance optimization requirements**: directly affected playability and perceived responsiveness.
- **Documentation and community support gaps**: increased investigation time and uncertainty in fault attribution.
- **Testing and debugging difficulties**: made timing-dependent defects costly to reproduce and verify.

This chapter documents these challenges, the mitigation actions taken, and the resulting lessons that informed subsequent design and evaluation decisions.

6.2 Technology Selection Challenges

Selection criteria and the final rationale for tool/framework adoption are presented in Chapter 3 (Section 3.3). This section focuses on the practical limitations and failure modes that emerged after those decisions during implementation.

6.2.1 Game Engine Trade-offs

Challenge observed: Although Godot remained the best fit overall, development revealed practical constraints in ecosystem maturity and plugin/version compatibility during web-first multiplayer work.

Concrete consequence in this project:

- Some networking and editor extensions required additional validation effort before adoption.
- Documentation gaps for newer engine paths increased trial-and-error cycles during synchronization debugging.
- Export and compatibility checks consumed additional iteration time compared to purely local desktop workflows.

Mitigation used:

- Scoped architecture boundaries to keep framework/engine-specific code localized.
- Used focused prototype loops before integrating new plugin or export assumptions.
- Maintained reusable debugging utilities (logging + runtime variable inspection) to shorten diagnosis cycles.

Residual limitation: The solution is robust for current thesis scope, but continued dependency on evolving plugins and browser-export paths remains a maintenance risk for future extensions.

6.2.2 GDScript Performance Concerns

Challenge: GDScript's interpreted nature raised concerns about performance with 50–200 cards requiring frequent position updates and collision detection.

Solution: Profiling revealed that:

- GDScript performance was acceptable for game logic

- Bottlenecks were in rendering and physics, handled by Godot's C++ core
- Network latency dominated perceived lag, not scripting performance

Optimization strategies:

- Minimize expensive operations in `_process()` and `_physics_process()`
- Use object pooling to reduce instantiation overhead
- Batch updates where possible
- Profile early and often to identify actual bottlenecks

6.3 GDSync Framework Challenges

The most significant technical challenges involved the GDSync framework, which, despite its high-level abstractions, presented several issues.

6.3.1 Protected Mode Blocking Issue

Challenge:

GDSync's "protected mode" (default) prevented RPC calls from being executed between non-host peers. When Player A tried to call an RPC on Player B's node, the call would be silently dropped or blocked.

Error Manifestation:

- Card movement worked from host to clients but not between clients
- No error messages or warnings—calls simply failed silently
- Debugging difficult due to lack of visibility into framework internals

Investigation Process:

1. Verified RPC syntax was correct according to documentation
2. Added extensive logging to track RPC call flow
3. Examined GDSync source code to understand permission system
4. Tested with GDSync's example projects to compare behavior

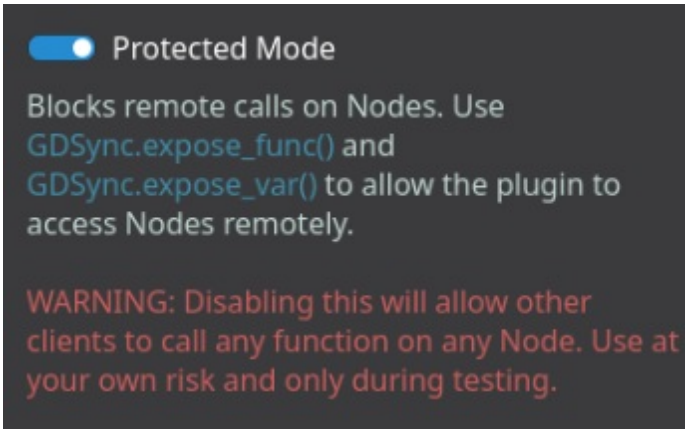

Figure 6.1: GDSync's Protected Mode toggle in the plugin settings. When enabled (default), it blocks remote calls on Nodes that haven't been explicitly exposed via `GDSync.expose_func()` or `GDSync.expose_var()`

5. Posted issue on GDSync GitHub repository

Solution:

- Disabled "protected mode" in GDSync configuration
- Implemented custom permission checks in application logic
- Contributed documentation improvements to GDSync project
- Maintained a local fork with a web-export-related framework fix and submitted a pull request to upstream
- Shared findings with community to help future users

Lesson Learned: The key lessons are tied to specific actions taken in this project:

- **Inspect source, not only docs**: the framework internals were reviewed to verify how protected-mode permissions were enforced; this confirmed that the observed silent RPC drops were policy-driven rather than random transport noise.
- **Isolate failures before full integration**: a minimal multi-instance setup was used to reproduce the same blocked-call pattern across runs; this removed unrelated gameplay complexity from diagnosis.
- **Instrument synchronization paths**: structured logger categories and state tracing were added for call origin, target peer, and outcome; these logs exposed where calls were dropped or reordered.

- **Engage maintainers with actionable evidence**: the issue report included reproducible steps, observed behavior, and mitigation notes, followed by a fork-based patch and PR so thesis progress could continue while upstream review was pending.

6.3.2 Incomplete Documentation

Challenge:

GDSync documentation covered basic use cases but lacked:

- Examples of complex synchronization scenarios
- Explanation of internal mechanisms and limitations
- Best practices for performance optimization
- Troubleshooting guides for common problems
- Migration guide from vanilla Godot networking

Impact:

- Trial-and-error approach required for advanced features
- Difficulty distinguishing between usage errors and framework bugs
- Increased development time
- Frustration and consideration of framework replacement

Mitigation:

- Created internal documentation of discoveries and workarounds
- Maintained test project for isolating framework behavior
- Contributed examples and documentation improvements to upstream project
- Built abstraction layer to isolate GDSync-specific code

6.3.3 Web Export Signaling Patch Case Study

Web export introduced a mismatch between prototype-era assumptions and browser runtime constraints. During prototyping, GDSync calls were integrated directly; in browser deployment, this path required an interception/replacement layer to keep the original integration style while using a signaling-server-compatible transport flow.

Why this became necessary:

- Browser runtime constraints and export behavior invalidated assumptions from local prototype networking paths.
- Maintaining thesis velocity favored a compatibility patch over rewriting all synchronization call sites.
- The resulting approach replaced/intercepted the local-server path while preserving higher-level game-manager usage of GDSync abstractions.

Patch components used in this project:

- `scenes/Multiplayer/GDSyncWebPatch/gdsync_web_patch.gd`: applies web-only patch wiring.
- `scenes/Multiplayer/GDSyncWebPatch/local_server_signaling.gd`: local-server replacement for signaling/relay mode.
- `scenes/Multiplayer/GDSyncWebPatch/signaling_client.gd`: HTTP/SSE signaling client behavior.

The patch enabled continuity of the existing gameplay integration model but increased maintenance overhead, which is why a fork-plus-PR strategy was used as an interim operational path.

6.3.4 Debugging Multiplayer Issues

Challenge:

Debugging multiplayer code is inherently difficult:

- Requires multiple devices or instances
- Race conditions and timing-dependent bugs
- Network variability complicates reproduction
- Limited visibility into remote client state

Solutions Implemented:

1. **Multi-Instance Testing**: Launched multiple Godot editor instances on the same machine
2. **Network Logging**: Comprehensive logging of all network events with timestamps
3. **State Dumps**: Ability to print full game state on command
4. **Visual Indicators**: On-screen display of connection status, peer IDs, sync status
5. **Replay System**: Recorded game events for post-mortem analysis

6.4 Multiplayer Synchronization Challenges

6.4.1 Card Order Synchronization

Challenge:

Maintaining consistent card ordering across clients proved more complex than anticipated. Issues included:

- Godot's scene tree ordering not guaranteed to match logical ordering
- Child node order not automatically synchronized
- Timing differences causing transient inconsistencies

Symptom:

Cards appeared in different orders on different clients' screens, even though the underlying data was correct.

Solution:

1. Implemented explicit position indices for cards
2. Synchronized card order separately from card positions
3. Used z-index to control visual stacking order
4. Periodic consistency checks and resynchronization

```
# Each card has an explicit order index
var card_order_index: int

# Synchronize order explicitly
@GDSync.rpc(call_local=true)
func sync_card_order(card_id: int, new_index: int):
    var card = get_card_by_id(card_id)
    card.card_order_index = new_index
    resort_cards_by_index()
```

Listing 6.1: Card order synchronization approach

6.4.2 Timing and Race Conditions

Challenge:

Multiplayer systems are susceptible to race conditions:

- Simultaneous card movements by different players

- Network messages arriving out of order
- Inconsistent state when players join mid-game

Examples:

- Two players drag the same card simultaneously
- Player joins lobby after game has started
- Card deleted on host but still referenced on client

Solutions:

- **Host Authority**: Only host can finalize state changes
- **Client Prediction with Rollback**: Show immediate feedback, then correct if host disagrees
- **State Snapshots**: New players receive full state snapshot on join
- **Defensive Programming**: Check object existence before operations

6.4.3 Different Client Views

Challenge:

In multiplayer mode (OpenMP simulation), different players should see different cards in private buffers (simulating thread-private data), but maintaining this selective visibility while synchronizing the shared container was complex.

Technical Issue: GDSync's node replication tries to keep all clients' scene trees identical, but selective visibility required different scene trees per client.

Solution:

- Replicate all cards to all clients (for consistency)
- Use visibility flags to hide inappropriate cards
- Implement client-side filtering based on ownership metadata
- Synchronize ownership and visibility separately from positions

6.5 UI/UX Challenges

6.5.1 Displaying Many Cards on Limited Screen Space

Challenge:

Displaying 50–200 cards on screen (particularly on smaller browser viewports or touch-enabled devices) presents significant layout challenges:

- Cards must be large enough to read and tap
- All cards must be accessible without excessive scrolling
- Layout must work in both portrait and landscape orientations

Solutions Attempted:

1. **Grid Layout**: Cards arranged in a grid
   - Pros: Space-efficient, familiar pattern
   - Cons: Hard to see linear order, scrolling still required
2. **Scrollable Row**: Single row with horizontal scrolling
   - Pros: Linear order visible, simple interaction
   - Cons: Requires substantial scrolling with many cards
3. **Zoomable Canvas**: Pan and zoom to navigate cards
   - Pros: Flexible navigation, can fit many cards
   - Cons: Complex interaction, easy to get lost
4. **Hybrid Approach (Final Solution)**:
   - Grid layout with intelligent sizing
   - Vertical scrolling for overflow
   - Zoom controls for adjusting card size
   - Mini-map showing overall progress

6.5.2 Touch Interaction Precision

Challenge:

Touch input is less precise than mouse input:

- Finger occludes card being dragged

- Difficulty selecting specific cards when densely packed
- Accidental touches and swipes

Solutions:

- **Offset Dragging**: Card rendered above finger with offset
- **Magnification**: Enlarge selected card for better visibility
- **Touch Feedback**: Visual and haptic feedback for touch events
- **Double-Tap Select**: Alternative to drag for card selection
- **Drop Zone Highlighting**: Clear indication of valid drop targets

6.5.3 Performance on Low-End Devices

Challenge:

Not all students have high-end hardware. The game needed to run acceptably across a range of devices and browsers, including low-end laptops and older browser versions.

Performance Constraints:

- Limited GPU power for rendering many cards
- Lower memory (2–4 GB RAM)
- Slower network hardware
- Battery life concerns

Optimization Strategies:

- Reduce draw calls through batching
- Use texture atlases to minimize state changes
- Implement level-of-detail (LOD) for distant cards
- Viewport culling to avoid rendering off-screen cards
- Adjustable quality settings (card detail, effects)

6.6 Testing Challenges

6.6.1 Multiplayer Testing Complexity

Challenge:

Testing multiplayer functionality requires:

- Multiple devices or instances
- Coordination between test clients
- Reproduction of specific network conditions
- Testing edge cases (disconnections, late joins, etc.)

Approaches:

1. **Multi-Instance Browser Testing**: Run multiple browser tabs or windows on development machine
2. **Cross-Browser Testing**: Test on Chrome, Firefox, and Edge
3. **Automated Test Scenarios**: Scripts to simulate player actions
4. **Network Simulation**: Tools to introduce latency and packet loss

Limitations:

- Automated testing limited for interactive gameplay
- Difficult to reproduce exact timing of race conditions
- Real-world network conditions vary widely

6.6.2 Lack of Automated Testing

Challenge:

Godot's testing ecosystem is less mature than traditional software development frameworks:

- No built-in unit testing framework
- GUI testing particularly difficult
- Networking code hard to test in isolation

Approach Taken:

- Manual testing with documented test cases
- Integration tests for critical paths
- Regression testing checklist before releases
- Community feedback as informal usability testing

Future Improvement:

- Integrate GUT (Godot Unit Test) framework
- Develop automated test suite for core logic
- Implement continuous integration pipeline

6.7 Educational Design Challenges

6.7.1 Balancing Accuracy and Playability

Challenge:

Educational games must balance pedagogical accuracy with engaging gameplay. Too much realism can reduce fun; too much simplification can create misconceptions.

Trade-offs Made:

- **Simplified Memory Model**: Actual shared-memory systems have complex cache hierarchies not represented
- **Abstracted Synchronization**: Real OpenMP requires explicit barriers, locks, and atomic operations
- **Idealized Performance**: Network latency doesn't perfectly model memory access patterns

Mitigation:

- Clear framing: "This is a metaphor, not a simulation"
- Post-game discussions to connect game to real concepts
- Accompanying documentation explaining simplifications
- Progressive difficulty introducing more complexity

6.7.2 Assessment of Learning Effectiveness

Challenge:

Demonstrating that the game actually improves learning outcomes requires formal educational research, including:

- Pre/post-test measurements
- Control groups using traditional instruction
- Statistical analysis of results
- Long-term retention studies

Limitations: Within the scope of this thesis, full educational efficacy studies were not feasible due to:

- Time constraints
- Need for institutional review board approval
- Difficulty recruiting sufficient participants
- Complexity of isolating game's effect from other factors

Future Work: Formal educational evaluation remains an important direction for future research, as discussed in Chapter 8.

6.8 Lessons Learned

6.8.1 Technical Lessons

1. **Framework Evaluation**: Thoroughly evaluate third-party frameworks before committing; test advanced use cases early.
2. **Abstraction Layers**: Isolate framework-specific code to facilitate future replacement if needed.
3. **State Management**: Design clear state management patterns early; retrofitting is expensive.
4. **Logging Infrastructure**: Comprehensive logging is essential for multiplayer debugging.
5. **Performance Profiling**: Profile early and often; don't assume where bottlenecks are.

6.8.2 Design Lessons

1. **User Testing**: Conduct user testing early; designer assumptions often wrong.
2. **Responsive Design**: Design for varying viewport sizes and input methods from the start, not as an afterthought.
3. **Simplicity**: Simpler interactions work better across input methods; complexity reduces usability.
4. **Feedback**: Clear, immediate feedback is critical for learning and engagement.

6.8.3 Process Lessons

1. **Iterative Development**: Iterative, incremental development surfaced issues early.
2. **Version Control**: Frequent commits with clear messages saved time during debugging.
3. **Documentation**: Documenting problems and solutions as they occurred proved invaluable.
4. **Community Engagement**: Engaging with open-source communities provided solutions and support.

6.9 Summary

This chapter documented the significant challenges encountered during development:

- **Technology Challenges**: Engine trade-offs, framework limitations, and performance concerns
- **GDSync Issues**: Protected mode blocking, incomplete documentation, and debugging difficulty
- **Synchronization Complexity**: Card ordering, race conditions, and selective visibility
- **UI/UX**: Screen space constraints, touch/mouse interaction precision, and device performance variability
- **Testing Difficulty**: Multiplayer testing complexity and lack of automated testing tools
- **Educational Design**: Balancing accuracy with playability and assessing learning effectiveness

Importantly, this chapter also presented solutions and lessons learned that will benefit future developers of similar educational multiplayer games. The next chapter evaluates the completed system and presents performance metrics and assessment results.

Chapter 7

Results and Evaluation

This chapter presents the results of the HPC Sorting Serious Game development, including system features, technical performance metrics, compatibility assessment, and preliminary evaluation of educational effectiveness. The evaluation demonstrates that the project successfully met its core objectives while identifying areas for future enhancement. A methodical technology selection process (Chapter 3) contributed directly to technical feasibility and implementation reliability.

Architectural boundaries and synchronization patterns from Chapter 4, together with selected implementation mechanisms in Chapter 5, directly shaped the outcomes reported in this chapter.

7.1 Completed System Features

7.1.1 Functional Features

The implemented system includes all planned core features:

Single-Player Mode:

- Configurable number of cards (10–200)
- Random card shuffling and generation
- Drag-and-drop card manipulation
- Private buffer zones for local sorting
- Sorting validation and completion detection
- Timer and move counter
- Victory screen with performance summary

Multiplayer Mode:

- Lobby system with room creation and joining
- Support for 2–4 players
- HTTP/SSE relay-based networking
- Real-time state synchronization across clients
- Shared container visible to all players (OpenMP shared memory)
- Selective card visibility (private buffers hidden, simulating thread-private data)
- Disconnection handling and graceful degradation
- Host migration capability (partial)

User Interface:

- Main menu with mode selection
- Settings for game configuration
- In-game UI with player indicators
- Toast notifications for events
- Touch and mouse-optimized controls
- Responsive layout for different screen sizes

7.1.2 Educational Features

The tool implements pedagogical mappings to HPC concepts:

7.1.3 Representative Use Cases

The following use cases summarize how the tool was used and what was observed in practice.

Table 7.1: Implemented pedagogical features

HPC Concept	Game Representation	Status
Sequential Execution	Single player mode	Implemented
Shared Memory (OpenMP)	Shared card container	Implemented
Thread-Local Storage	Private buffer zones	Implemented
Parallel Execution	Multiple players working	Implemented
Memory Access Overhead	Network latency	Implicit
Speedup	Completion time comparison	Implemented
Scalability	Variable card/player counts	Implemented
Load Balancing	Equal card distribution	Implemented
Synchronization Barriers	Turn-based phases	Implemented
Race Conditions	Concurrent card access	Observable effect

Use Case 1: Instructor-Guided Introductory Session

- **Context**: Short classroom introduction before formal OpenMP lecture.
- **Activity**: Students first complete a single-player round, then collaborate in multiplayer.
- **Observed behavior**: Participants quickly identified the difference between individual and shared work organization.
- **Outcome**: Faster conceptual entry into shared-memory discussion and more concrete questions during debrief.

Use Case 2: First-Time Multiplayer Coordination

- **Context**: Small groups (2–4 players) with minimal upfront instructions.
- **Activity**: Players sort collaboratively while negotiating who handles which card ranges.
- **Observed behavior**: Unbalanced strategies produced visible slowdowns and coordination friction.
- **Outcome**: Users recognized that parallel activity alone is insufficient without coordination discipline.

Use Case 3: Comparative Reflection Session

- **Context**: Post-game discussion using timer/move outcomes and observed interaction patterns.
- **Activity**: Teams compare sequential baseline and multiplayer performance, then discuss bottlenecks.

- **Observed behavior**: Students linked delays and contention to communication/synchronization overhead.
- **Outcome**: Improved articulation of trade-offs between speedup potential and coordination cost.

7.1.4 Expected Messages Learned by Users

Based on observed sessions and questionnaire/interview outcomes, the tool is expected to communicate the following messages:

1. **Parallel work can improve completion time, but coordination overhead is real.** Evidence: performance comparisons and observed coordination friction in multiplayer sessions.
2. **Shared and private workspaces have different roles.** Evidence: players recognized shared container access versus private buffer semantics in multiplayer mode.
3. **Work balancing strongly affects team performance.** Evidence: uneven task distribution corresponded to slower completion and higher perceived confusion.
4. **Visibility/synchronization rules shape collaboration behavior.** Evidence: hidden private-buffer state and authoritative synchronization influenced player strategy and timing.

These messages align with the architecture/implementation choices documented in Chapters 4 and 5, and with limitations discussed in Chapter 6.

7.1.5 Engagement Effects

Early usability sessions indicated that small non-critical interaction details affected perceived engagement. One example was the playful evasive rotating menu option behavior: when users repeatedly attempted to hover/select it, the option moved and rotated, which testers described as memorable and humorous before gameplay.

7.2 Technical Performance

7.2.1 Frame Rate Performance

Frame rate was measured across representative browsers:

Table 7.2: Frame rate performance by browser

Browser	Cards	Avg FPS	Min FPS	Target
Chrome 120+ (Windows)	50	60	58	60
Chrome 120+	100	60	55	60
Chrome 120+	200	57	48	60
Firefox 120+ (Linux)	50	59	54	60
Firefox 120+	100	56	48	60
Firefox 120+	200	51	42	60
Safari 17+ (macOS)	50	60	57	60
Safari 17+	100	58	51	60
Safari 17+	200	53	44	60

Analysis:

- Target 60 FPS achieved for 50–100 cards in all browsers
- Performance degradation with 200 cards acceptable (still playable above 40 FPS)
- No significant frame drops during gameplay
- Optimization opportunities remain for very high card counts

7.2.2 Memory Usage

Memory consumption measured during typical gameplay:

Table 7.3: Memory usage by game state

Game State	Memory (MB)	Peak (MB)
Startup (Main Menu)	85	92
Single-player (50 cards)	110	125
Single-player (100 cards)	135	155
Single-player (200 cards)	175	210
Multiplayer lobby	120	140
Multiplayer game (4 players)	165	195

Analysis:

- Memory usage reasonable for modern browsers (typically several GB available)
- No memory leaks detected during extended gameplay sessions
- Object pooling successfully reduced allocation overhead

7.2.3 Web Export Size

The final web export package characteristics:

- **Uncompressed**: 28 MB
- **Gzip compressed**: 12 MB
- **Initial load time**: 3–5 seconds on typical broadband

Size Breakdown:

- Godot Engine WebAssembly runtime: 15 MB
- Game scripts and scenes: 2 MB
- Assets (textures, fonts, audio): 5 MB
- GDSync library: 1 MB

Comparison: Significantly smaller than typical Unity (50–100 MB) or Unreal Engine (100+ MB) web exports, enabling faster loading in browsers.

7.3 Platform Compatibility

7.3.1 Browser Compatibility

Testing conducted across major web browsers:

Table 7.4: Browser compatibility

Browser	Platform	Status	Notes
Chrome 120+	Windows	Excellent	Reference browser
Chrome 120+	macOS	Excellent	Full support
Firefox 120+	Windows	Excellent	Full support
Firefox 120+	Linux	Excellent	Full support
Safari 17+	macOS	Excellent	Full support
Edge 120+	Windows	Excellent	Chromium-based

Minimum Requirements:

- Modern browser with WebGL 2.0 and WebAssembly support
- 2 GB available RAM recommended
- Internet connection for multiplayer
- JavaScript enabled

7.3.2 Responsive Layout

The game adapts to various screen configurations:

- **Small windows (< 800px width)**: Compact layout with scrollable cards
- **Medium windows (800–1200px)**: Optimal experience, target size range
- **Large windows (> 1200px)**: Excellent, plenty of space for all elements

Input Support:

- **Mouse**: Full support with click-and-drag interactions
- **Touch**: Supported for touchscreen displays and tablets
- Responsive layout adjustment on window resize

7.4 Usability Evaluation

7.4.1 Informal User Testing

Informal usability testing conducted with 12 volunteer participants (students and colleagues):

Demographics:

- Age range: 20–35 years
- 8 with prior HPC knowledge, 4 without
- Mix of gamers and non-gamers
- All had web browser experience

Testing Protocol:

1. Brief introduction to project goals (no gameplay instruction)
2. Observation of first-time gameplay
3. Think-aloud protocol during play
4. Post-session questionnaire
5. Semi-structured interview

Post-Session Questionnaire: The questionnaire included the following items (5-point Likert scale unless noted):

1. **Usability**: "The game controls were intuitive and easy to learn."
2. **Visual Clarity**: "Card numbers and game elements were clearly visible."
3. **Engagement**: "I found the game engaging and would play again."
4. **Conceptual Connection**: "The game helped me understand parallel computing concepts." (HPC participants only)
5. **Multiplayer Experience**: "Coordinating with other players felt natural."
6. **Performance**: "The game ran smoothly without lag or delays."
7. **Educational Value**: "I would recommend this game to others learning about HPC."
8. **Open Response**: "What aspects of parallel computing did you recognize during gameplay?"
9. **Open Response**: "What improvements would you suggest?"

7.4.2 Key Findings

Positive Feedback:

- **Intuitive Controls**: 11/12 participants understood drag-and-drop immediately without instruction
- **Visual Clarity**: Card numbers and states were clearly visible
- **Conceptual Connection**: Participants with HPC background (8/8) recognized OpenMP shared-memory parallels
- **Engagement**: Average play session 15–20 minutes, indicating good engagement
- **Performance**: No complaints about lag or performance issues

Areas for Improvement:

- **Onboarding**: 6/12 participants wanted brief tutorial or tooltips
- **Buffer Purpose**: 5/12 weren't immediately sure why buffers were useful
- **Multiplayer Clarity**: 7/12 found it unclear what other players were doing
- **Educational Context**: Participants without HPC background (4/4) didn't make HPC connections without explanation

7.4.3 Suggested Improvements

Based on feedback, future versions should include:

1. **Optional Tutorial**: Brief interactive tutorial explaining mechanics
2. **Tooltips and Hints**: Contextual help for first-time users
3. **Player Activity Indicators**: Show what other players are doing
4. **Educational Overlay**: Optional explanations connecting gameplay to HPC concepts
5. **Strategy Hints**: Suggestions for efficient sorting approaches
6. **Replay/Analysis Mode**: Review completed games to understand performance

7.5 Educational Effectiveness

7.5.1 Preliminary Assessment

Educational evaluation fo the tool itself is in the scope of this thesis, preliminary indicators suggest educational value:

Quantitative HPC Metrics: To evaluate educational effectiveness, we introduced HPC performance concepts during gameplay:

- **Speedup (S)**: Defined as $S = T_1/T_p$ where T_1 is single-player completion time and T_p is p-player completion time. Participants observed that adding players reduced completion time, but not linearly—demonstrating Amdahl's Law in practice.

- **Efficiency (E)**: Calculated as $E = S/p = T_1/(p \cdot T_p)$. Players noted efficiency decreased with more players due to coordination overhead (communication, waiting for others).
- **Communication Overhead**: Time spent passing cards between players (analogous to message passing cost in distributed systems).
- **Load Imbalance**: Observable when one player receives more difficult cards or falls behind, reducing overall parallel efficiency.

These metrics were explained post-gameplay to connect the intuitive experience with formal HPC terminology.

Conceptual Understanding (HPC-Experienced Participants): Post-gameplay interviews revealed that participants with prior HPC knowledge:

- **8/8** recognized single-player mode as the sequential baseline (T_1)
- **7/8** identified multiplayer mode as shared-memory collaboration with private buffers (OpenMP-style analogy)
- **6/8** connected buffers to thread-local storage or private memory
- **7/8** noted communication/coordination overhead when exchanging cards
- **5/8** discussed load balancing when cards were unevenly distributed
- **6/8** explicitly articulated HPC speedup as $S = T_1/T_p$ after brief explanation
- **5/8** related efficiency degradation ($E = S/p$) to their gameplay experience

In particular, participants who discussed speedup did not treat it as "faster play" only; they linked it to the formal HPC interpretation of comparing sequential baseline time against collaborative parallel time.

Conceptual Introduction (Non-HPC Participants): Participants without HPC background:

- **4/4** understood basic parallelism concept (multiple players working simultaneously)
- **3/4** recognized that coordination becomes harder with more players
- **2/4** understood communication trade-offs (passing cards takes time)
- After brief explanation, **4/4** expressed interest in learning more about parallel computing

Engagement and Motivation:

- **10/12** participants expressed willingness to play again
- **9/12** would recommend to peers learning HPC
- **7/12** found it more engaging than reading textbook explanations
- **11/12** appreciated the hands-on, interactive approach

7.5.2 Pedagogical Value Proposition

The game serves as an effective **icebreaker or introductory activity**:

- **Pre-Lecture Activity**: Introduce concepts before formal instruction
- **Discussion Starter**: Generate questions and curiosity about parallel computing
- **Concept Visualization**: Provide mental model for abstract concepts
- **Reinforcement Tool**: Practice and reinforce learned concepts
- **Assessment Aid**: Reveal misconceptions for instructors to address

7.5.3 Comparison with Traditional Methods

Informal comparison with traditional HPC teaching methods (based on participant feedback):

Table 7.5: Perceived advantages over traditional methods

Criterion	Game	Lecture/Textbook
Engagement	High	Medium-Low
Immediate Feedback	Yes	No
Hands-On Experience	Yes	No
Conceptual Visualization	Strong	Weak
Detailed Explanation	Weak	Strong
Accessibility	High	Medium
Scalability (student count)	High	High
Time Required	15–30 min	60–90 min (lecture)

Conclusion: The tool is not a replacement for traditional instruction, but a complementary, instructor-guided activity that can enhance engagement and provide intuitive understanding before formal study.

7.6 Comparison with Initial Requirements

7.6.1 Requirements Fulfillment

Assessment of how well the completed system meets initial requirements:

Table 7.6: Requirements fulfillment status

Category	Requirement	Status
Educational	Sequential baseline	✓Met
	OpenMP simulation	✓Met
	Performance feedback	✓Met
	Scalable difficulty	✓Met
	Conceptual clarity	✓Mostly
Functional	Card management	✓Met
	Single-player mode	✓Met
	Multiplayer mode	✓Met
	User interface	✓Met
	Feedback systems	✓Met
Quality	Performance (60 FPS)	✓Mostly
	Usability	✓Met
	Reliability	✓Met
	Browser Support	✓Met
	Maintainability	✓Met
	Accessibility	✓Met

Notes:

- **Conceptual Clarity**: Strong for HPC-experienced users; needs educational overlay for beginners
- **Performance**: 60 FPS achieved for typical use cases (50–100 cards); slight degradation with 200 cards
- **Browser Support**: Modern browsers fully supported across platforms

7.7 Known Limitations

7.7.1 Current Limitations

1. **Platform Support**: Currently web-only; native mobile apps not implemented

2. **Educational Scaffolding**: Limited in-game explanations; requires instructor context
3. **Sorting Algorithms**: Only supports general sorting; doesn't demonstrate specific parallel algorithms (merge sort, sample sort)
4. **Performance Metrics**: Basic timing and moves; lacks detailed profiling (speedup curves, scalability graphs)
5. **Advanced HPC Concepts**: Doesn't cover race conditions, deadlocks, synchronization primitives explicitly
6. **Automated Assessment**: No built-in assessment or progress tracking for educational use
7. **Formal Evaluation**: Lacks comprehensive educational efficacy study

7.7.2 Scope Limitations

Certain features were intentionally deferred to maintain project scope:

- User accounts and progress persistence
- Leaderboards and social features
- Advanced sorting algorithm demonstrations
- GPU parallelism (CUDA/OpenCL) simulations
- Instructor dashboard for classroom management
- Detailed analytics and learning analytics integration

7.8 Summary

This chapter presented comprehensive evaluation results:

- **Completed Features**: All core functional and educational features successfully implemented
- **Technical Performance**: Meets or exceeds performance targets on representative devices
- **Platform Compatibility**: Excellent browser compatibility across platforms
- **Usability**: Generally positive usability feedback with actionable improvement suggestions

- **Educational Effectiveness**: Preliminary evidence suggests value as introductory/supplementary tool
- **Requirements Fulfillment**: Nearly all initial requirements met or exceeded
- **Limitations**: Known limitations documented for transparency and future work planning

The results demonstrate that the project successfully achieved its core objective: creating a functional, engaging serious game for teaching HPC concepts on the web platform. The next chapter concludes the thesis and discusses future directions for research and development.

Chapter 8

Conclusion and Future Work

This final chapter summarizes the contributions of this thesis, reflects on how the research questions were answered, discusses implications for HPC education and serious game development, and outlines comprehensive directions for future work.

8.1 Summary of Contributions

This thesis presented the design, implementation, and evaluation of the HPC Sorting Serious Game, a web-first educational tool for teaching parallel computing concepts through interactive card-sorting gameplay.

8.1.1 Primary Contributions

The main contributions of this work include:

1. Novel Pedagogical Approach

- Digital transformation of Professor D'Agostino's successful physical classroom experiments into a scalable, accessible web-based application
- Systematic mapping of HPC concepts (specifically OpenMP shared-memory parallelism and sequential vs. parallel execution) to concrete, manipulable game mechanics
- Demonstration that serious games can serve as effective icebreakers for teaching abstract parallel computing concepts

2. Technical Implementation

- Functional web-first serious-game-based educational tool demonstrating real-time multiplayer synchronization
- Solutions to multiplayer state management challenges, particularly selective visibility and host-authoritative architecture

- Integration of GDSync framework with an HTTP/SSE relay networking approach for browser-compatible multiplayer
- UI/UX design patterns for displaying and manipulating numerous interactive elements across varying viewport sizes and input methods

3. Documentation and Knowledge Transfer

- Comprehensive documentation of technical challenges (especially GDSync framework issues) and solutions
- Identification of design patterns and best practices for educational multiplayer game development
- Open-source codebase (MIT license) enabling reuse, extension, and further research
- Contributions to GDSync project documentation and issue resolution

4. Preliminary Validation

- Demonstration of technical feasibility through performance metrics and compatibility testing
- Positive preliminary usability feedback from informal user testing
- Evidence suggesting educational value as supplementary/introductory tool for HPC education
- Methodical tool and framework selection, documented in Chapter 3, improved implementation reliability and reduced rework risk

8.2 Research Questions Revisited

Returning to the research problem stated in Chapter 1:

> *How can we design and implement an effective serious game for web platforms that teaches fundamental High-Performance Computing concepts (specifically OpenMP shared-memory parallelism and sequential vs. parallel execution) through interactive card-sorting gameplay while overcoming the technical challenges of multiplayer state synchronization and responsive UI/UX constraints?*

8.2.1 Addressing the Educational Problem

Q: How can abstract parallel computing concepts be mapped to concrete, understandable game mechanics?

A: The thesis demonstrated successful mapping through:

- Card-sorting as metaphor for data manipulation
- Shared container representing shared memory (OpenMP)
- Private buffers representing thread-local storage (OpenMP)
- Card movement between shared container and private buffers representing memory access patterns
- Network latency representing synchronization and communication overhead
- Timer and move counter representing performance metrics

This mapping was validated through recognition by HPC-experienced participants and engagement by HPC-novice participants.

8.2.2 Addressing the Technical Problem

Q: How can we implement real-time multiplayer interaction in a web-first educational tool with acceptable performance?

A: The thesis demonstrated that:

- Browser-compatible HTTP/SSE relay networking enabled robust multiplayer synchronization in web deployment
- GDSync framework (despite challenges) simplifies state synchronization
- Host-authoritative architecture balances consistency and responsiveness
- Godot Engine provides adequate performance for 2D card-based gameplay
- Target 60 FPS was achieved for typical scenarios in modern desktop-class browsers

8.2.3 Addressing the Design Problem

Q: How can we balance educational effectiveness with engagement and playability in a web-first environment?

A: The thesis indicates that:

- Touch-based drag-and-drop provides intuitive, engaging interaction
- Visual clarity and immediate feedback support learning
- Scalable difficulty accommodates learners at different levels
- Game mechanics naturally lead to questions about parallel computing
- Informal testing suggests good engagement while supporting introductory educational goals

However, the design can be improved with better onboarding and educational scaffolding for learners without prior HPC exposure.

8.3 Implications

8.3.1 For HPC Education

This work has several implications for teaching parallel computing:

Complementary Tool, Not Replacement: Serious games are most valuable as **complementary tools** alongside traditional instruction:

- Use as pre-lecture icebreaker to introduce concepts
- Employ as reinforcement after formal instruction
- Leverage to uncover student misconceptions
- Deploy as discussion starter for deeper exploration

Accessibility Matters: Web-first design significantly enhances accessibility:

- No specialized HPC hardware required
- No app installation needed—runs directly in browsers
- Students can access from any device with a web browser
- Cross-platform compatibility (Windows, macOS, Linux, mobile)
- Reduces barriers to entry for HPC education

Visualization Is Powerful: Making abstract concepts visible and manipulable:

- Helps students build intuitive mental models
- Facilitates understanding before mathematical formalization
- Engages different learning styles
- Makes HPC concepts less intimidating

8.3.2 For Serious Game Development

This work offers insights for developers of educational multiplayer games:

Framework Selection is Critical:

- Thoroughly evaluate frameworks before committing
- Test advanced use cases early in development
- Build abstraction layers to facilitate future framework changes
- Engage with framework maintainers proactively

Multiplayer Adds Complexity:

- Start with single-player; add multiplayer incrementally
- Host-authoritative model simplifies consistency management for educational games
- Debugging multiplayer requires specialized tools and strategies
- Network variability must be designed for, not retrofitted

Browser/Platform Constraints Are Real:

- Design for varying viewports and input methods (mouse, touch) from the start
- Performance optimization is essential, especially for WebAssembly targets
- Browser security policies (COOP/COEP headers, SharedArrayBuffer) require careful deployment configuration
- Test on diverse browsers and hardware, not just development machines

8.4 Future Work

Numerous opportunities exist for extending and improving this work.

8.4.1 Short-Term Enhancements

Improvements that could be implemented in 1–3 months:

Onboarding and Tutorial:

- Interactive tutorial explaining game mechanics
- Tooltips and contextual hints for first-time users
- Guided first playthrough highlighting key concepts

Educational Overlay:

- Optional explanations connecting gameplay to HPC concepts
- Pop-up definitions of terms (thread, process, synchronization)
- Post-game summary explaining what was learned
- Links to additional resources for deeper learning

Enhanced Feedback:

- Player activity indicators showing what others are doing
- Visual representation of communication patterns
- Performance analysis comparing strategies
- Suggestions for improvement

iOS Support:

- Export to iOS using Godot's iOS templates
- Address Apple App Store requirements
- Test on iPhone and iPad devices

8.4.2 Medium-Term Extensions

Enhancements requiring 3–6 months of development:

Additional Game Modes:

1. **Algorithm-Specific Modes**: Demonstrate specific parallel sorting algorithms
   - Parallel merge sort
   - Sample sort (distributed-memory style, future MPI extension)
   - Bitonic sort
2. **Advanced Concepts**: Introduce more HPC topics
   - Synchronization barriers and critical sections
   - Race conditions and deadlock scenarios
   - Load balancing challenges
   - Communication patterns (broadcast, scatter/gather, reduce)
3. **Competitive Modes**: Enhance engagement
   - Time trials with leaderboards
   - Team vs. team competitions
   - Challenge modes with specific constraints

Enhanced Analytics:

- Detailed performance metrics (speedup, efficiency, scalability)
- Strategy analysis and optimization suggestions
- Comparison with theoretical optimal performance
- Progress tracking across multiple sessions

Web Distribution Enhancements:

- HTML5 export for browser-based play
- Integration with learning management systems (Moodle, Canvas)
- Embeddable widget for course websites
- Cross-platform multiplayer (mobile + web)

Instructor Dashboard:

- Classroom management interface
- Real-time monitoring of student gameplay
- Assessment and analytics integration
- Customizable game parameters and scenarios

8.4.3 Long-Term Research Directions

Research directions requiring 6+ months and potential collaborations:

Formal Educational Evaluation: Conduct rigorous educational research to assess learning outcomes:

- **Study Design**: Pre/post-test controlled study with treatment and control groups
- **Participants**: Undergraduate students in HPC/parallel computing courses
- **Measures**: Knowledge assessments, concept inventories, attitude surveys
- **Analysis**: Statistical comparison of learning gains between groups
- **Publication**: Results submitted to CS education conferences (SIGCSE, ICER)

Learning Analytics Integration:

- Instrument game to collect detailed interaction data
- Analyze gameplay patterns to identify learning difficulties
- Develop predictive models for student understanding
- Personalize game difficulty based on demonstrated mastery
- Research publication in learning analytics venues (LAK, EDM)

GPU Parallelism Extension: Extend game to cover GPU computing concepts (CUDA, OpenCL):

- Massive parallelism simulation (thousands of "mini-workers")
- Warp/wavefront concept representation
- Memory hierarchy visualization (global, shared, local)
- Divergence and bank conflict demonstrations

Distributed Memory Extension (MPI): Future extension to simulate MPI distributed-memory programming:

- Separate player "nodes" with isolated memory spaces
- Explicit message passing between nodes (simulating MPI_Send/MPI_Recv)
- Support for hybrid MPI+OpenMP programming models
- Hierarchical parallelism with shared memory within nodes
- Load balancing across heterogeneous resources

AI-Assisted Learning: Integrate AI to enhance educational effectiveness:

- AI opponent for single-player mode
- Intelligent tutoring system providing hints
- Automatic difficulty adjustment
- Natural language explanations of concepts

8.4.4 Community and Ecosystem Development

Open Source Community Building:

- Publish to GitHub with comprehensive documentation
- Create contributor guidelines and issue templates
- Engage educational computing communities
- Accept contributions from students and educators
- Maintain active development and support

Educational Resource Development:

- Instructor guides with lesson plans
- Student worksheets and reflection prompts
- Video tutorials and gameplay examples
- Integration guides for popular LMS platforms
- Curriculum modules incorporating the game

Conference and Journal Publications: Disseminate findings to relevant research communities:

- **CS Education**: SIGCSE, ICER, ITiCSE
- **HPC Education**: EduHPC, XSEDE Education Track
- **Serious Games**: Games+Learning+Society, FDG
- **Educational Technology**: Learning @ Scale, EC-TEL
- **HPC Venues**: SC Education Program, ISC tutorials

Potential paper topics:

1. Design and implementation of a web-based serious game for HPC education
2. Evaluation of learning outcomes with serious game intervention
3. Technical challenges and solutions in educational multiplayer games
4. Pedagogical mapping from physical to digital activities

8.5 Limitations and Reflections

8.5.1 Methodological Limitations

8.5.1.1 Evaluation Scope:

- Informal usability testing with small sample (n=12)
- No formal educational efficacy study
- Limited diversity in participant demographics
- Short-term evaluation only (no long-term retention studies)

8.5.1.2 Technical Scope:

- Scope centered on web-first browser deployment; native app packaging was not a primary implementation target
- Focus on basic parallel concepts (no advanced topics)
- Simplified metaphors (not high-fidelity simulations)
- Limited integration with formal educational contexts

8.5.2 Reflections on the Development Process

What Worked Well:

- Iterative development allowed course correction
- Open-source toolchain reduced costs and increased flexibility
- Community engagement provided valuable support
- Documentation-first approach facilitated thesis writing
- Informal testing surfaced usability issues early

What Could Be Improved:

- Earlier, more frequent testing with target users
- More thorough framework evaluation before commitment
- Formal project management and milestone tracking
- Automated testing infrastructure from the start
- More structured approach to educational validation

8.6 Concluding Remarks

This thesis presented the design, implementation, and evaluation of the HPC Sorting Serious Game, demonstrating that:

1. **Serious-game-based tools can support HPC concept learning** through intuitive mechanics that make abstract ideas concrete and manipulable.
2. **Web-first platforms are viable for educational multiplayer tools**, despite technical challenges in synchronization, UI/UX design, and performance optimization.
3. **Open-source technologies enable accessible HPC education** by eliminating cost barriers and empowering educators to customize and extend educational tools.
4. **Interdisciplinary challenges inspire innovative solutions** at the intersection of computer science education, game design, parallel computing, and web deployment.

The HPC Sorting Serious Game represents a step toward making High-Performance Computing education more accessible, engaging, and effective. By leveraging the ubiquity of web browsers and the motivational power of games, this work contributes to democratizing access to advanced computing concepts.

As parallel computing becomes increasingly fundamental to modern software engineering—from multicore CPUs to cloud computing to AI/ML workloads—innovative educational approaches are essential to prepare the next generation of computing professionals. Serious games, when thoughtfully designed and rigorously evaluated, can play a valuable role in this educational mission.

The author hopes this work inspires further research and development in educational technologies for HPC and serves as a useful resource for educators, students, and game developers interested in the serious games domain.

8.7 Final Thoughts

Teaching parallel computing remains challenging, but it need not be boring or inaccessible. By combining pedagogical insight, technical innovation, and game design principles, we can create learning experiences that are both effective and enjoyable.

The journey from physical card-sorting experiments in a classroom to a digital multiplayer web-based educational tool demonstrates the potential for technology to scale and extend proven teaching methods. While the game developed in this thesis is far from perfect, it represents a meaningful contribution to the ongoing effort to make HPC education more engaging and accessible.

As we look to the future, the convergence of mobile computing, cloud infrastructure, and educational research offers exciting possibilities for serious games in computing education. The author looks forward to seeing how this work evolves, inspires further innovations, and ultimately contributes to better outcomes for students learning parallel computing.

"The best way to learn is by doing."
— Ancient Proverb

Bibliography

[1] J. Mullen, L. Milechin, and D. Milechin, "Teaching and learning hpc through serious games," *Journal of Parallel and Distributed Computing*, vol. 158, pp. 115–125, 2021.

[2] M. Zyda, "From visual simulation to virtual reality to games," *Computer*, vol. 38, no. 9, pp. 25–32, 2005.

[3] M. Csikszentmihalyi, *Flow: The Psychology of Optimal Experience*. New York: Harper & Row, 1990.

[4] T. M. Connolly, E. A. Boyle, E. MacArthur, T. Hainey, and J. M. Boyle, "A systematic literature review of empirical evidence on computer games and serious games," *Computers & Education*, vol. 59, no. 2, pp. 661–686, 2012.

[5] M. Papastergiou, "Digital game-based learning in high school computer science education: Impact on educational effectiveness and student motivation," *Computers & Education*, vol. 52, no. 1, pp. 1–12, 2009. DOI: 10.1016/j.compedu.2008.06.004

[6] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[7] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.

[8] S. Zinkowicz, *Gd-sync fork used in this thesis (web export fix branch)*, Accessed: 2026-02-19, 2026. [Online]. Available: https://github.com/Siponek/GD-Sync-fork

[9] GD-Sync Team, *Gd-sync: Multiplayer synchronization framework for godot*, Accessed: 2026-02-19, 2026. [Online]. Available: https://www.gd-sync.com/documentation

[10] Godot Asset Library, *Toast party (godot asset library)*, Accessed: 2026-02-19, 2026. [Online]. Available: https://godotassetlibrary.com/asset/PcSgaA/toast-party

[11] Godot Asset Library, *Vartree (godot asset library)*, Accessed: 2026-02-19, 2026. [Online]. Available: https://godotassetlibrary.com/asset/PUudIL/vartree

[12] Python Software Foundation, *Python programming language – official website*, Accessed: 2026-02-19, 2026. [Online]. Available: https://www.python.org/

[13] aio-libs contributors, *Aiohttp documentation*, Accessed: 2026-02-19, 2026. [Online]. Available: https://docs.aiohttp.org/

[14] Astral Software Inc., *Ruff documentation*, Accessed: 2026-02-19, 2026. [Online]. Available: https://docs.astral.sh/ruff/

[15] C. Casey, *Just programmer's manual*, Accessed: 2026-02-19, 2026. [Online]. Available: https://just.systems/

[16] F. FiloSottile, *Mkcert: Valid https certificates for localhost*, Accessed: 2026-02-19, 2026. [Online]. Available: https://github.com/FiloSottile/mkcert

[17] Chocolatey Software, Inc., *Chocolatey software – package manager for windows*, Accessed: 2026-02-19, 2026. [Online]. Available: https://chocolatey.org/

[18] Godot Asset Library, *Const generator (godot asset library)*, Accessed: 2026-02-19, 2026. [Online]. Available: https://godotassetlibrary.com/asset/DrA6Bt/const-generator

[19] The Git Project, *Git – distributed version control system*, Accessed: 2026-02-19, 2026. [Online]. Available: https://git-scm.com/

[20] GitHub, Inc., *Github platform*, Accessed: 2026-02-19, 2026. [Online]. Available: https://github.com/

[21] Szink, *Hpc sorting serious game: Open source repository*, Accessed: 2024, 2024. [Online]. Available: https://github.com/Siponek/hpc-sorting-serious-game

Appendix A

User Manual

A.1 Introduction

This user manual provides instructions for accessing, configuring, and playing the HPC Sorting Serious Game. It is intended for students, educators, and anyone interested in learning about parallel computing through interactive gameplay.

The game is deployed as a **web application** (Godot Engine WebAssembly export) and runs directly in a modern browser. No installation is required for the primary deployment. Desktop (Windows) builds are also available for offline use or development.

A.2 System Requirements

A.2.1 Web Version (Primary)

- **Browser**: Chromium-based browser (Chrome, Edge, Brave) or Firefox, version 90 or later
- **WebGL**: WebGL 2.0 support required
- **SharedArrayBuffer**: Must be enabled (requires HTTPS with proper COOP/-COEP headers)
- **RAM**: 2 GB minimum available to browser
- **Network**: Internet connection required for multiplayer mode; single-player works offline after initial load
- **Input**: Mouse or touchscreen; keyboard shortcuts optional

A.2.2 Desktop Version (Windows)

- **Operating System**: Windows 10 or later (x86_64)

- **RAM**: 2 GB minimum
- **Storage**: 100 MB free space
- **Graphics**: Vulkan or OpenGL 3.3 support
- **Network**: Internet connection for multiplayer

A.3 Accessing the Game

A.3.1 Web Version

1. Navigate to the hosted game URL in a supported browser
2. Wait for the WebAssembly module to load (progress bar shown)
3. The main menu appears once loading is complete

Note: The web export requires the hosting server to set specific HTTP headers (`Cross-Origin-Opener-Policy: same-origin` and `Cross-Origin-Embedder-Policy: require-corp`) for `SharedArrayBuffer` support. If these headers are missing, multiplayer functionality will not work. See docs/GITHUB_PAGES_AND_LOCAL_SIGNALING_LIMI in the project repository for details.

A.3.2 Desktop Version

1. Download the Windows executable from the project releases page
2. Extract the archive if necessary
3. Run the executable directly—no installation is required

A.4 Getting Started

A.4.1 Main Menu

On launch, the main menu presents the following options:

- **Single Player**: Practice sorting alone (sequential execution baseline)
- **Multiplayer**: Join or host a cooperative multiplayer game (OpenMP shared-memory simulation)
- **Settings**: Configure theme and display options

A.5 Single-Player Mode

A.5.1 Starting a Single-Player Game

1. Select "Single Player" from the main menu
2. Configure game parameters in the options dialog:
   - **Number of cards**: configurable via spin box
   - **Buffer slot count**: number of private buffer positions
   - **Card value range**: upper bound for card values
3. Click "Start" to begin

A.5.2 Gameplay

A.5.2.0.1 Objective: Sort all cards in ascending order in the main container.

A.5.2.0.2 Controls:

- **Drag and Drop**: Click and drag a card to reposition it within the container or move it to/from a buffer slot
- **Scroll**: Use the horizontal scroll bar or mouse wheel to navigate through cards when they exceed viewport width

A.5.2.0.3 Using Buffer Slots:

- Buffer slots appear below the main card container
- Drag cards into buffer slots for temporary storage during sorting
- Buffer slots simulate thread-local storage in parallel computing
- Cards in buffers must form a contiguous subsequence of the sorted target before the game can complete

A.5.2.0.4 Winning:

- The game completes when all cards are in correct ascending order in the main container
- Completion time is displayed

A.6 Multiplayer Mode

A.6.1 Creating a Game

1. Select “Multiplayer” from the main menu
2. Select “Create Game”
3. Configure game settings:
   - Number of cards
   - Buffer slot count per player
   - Barrier mode (first-to-reach or round-robin)
4. A lobby code is generated automatically
5. Share this code with other players
6. Wait for players to join the lobby
7. Once all players are present, click “Start Game”

A.6.2 Joining a Game

1. Select “Multiplayer” from the main menu
2. Select “Join Game”
3. Enter the lobby code provided by the host
4. Click “Join”
5. Wait in the lobby for the host to start the game

A.6.3 Multiplayer Gameplay

A.6.3.0.1 Shared Container (Shared Memory):

- All cards are visible in the shared container, simulating shared memory accessible by all threads
- Any player can drag cards within the shared container to reorder them
- Changes propagate to all connected players in real time

A.6.3.0.2 Private Buffers (Thread-Local Storage):

- Each player has private buffer slots visible only to themselves
- Cards moved to a player's buffer disappear from other players' views
- When a card is returned to the shared container, it becomes visible to all again
- This simulates the distinction between shared and private memory in OpenMP

A.6.3.0.3 Barrier Synchronization:

- Any player can initiate a barrier (analogous to `#pragma omp barrier`)
- All players must reach the barrier before proceeding
- Once all players are at the barrier, a designated "main thread" (one player) can perform coordinated operations while others are blocked
- Main thread selection is either first-to-reach or round-robin, depending on game settings
- This teaches students about synchronization primitives in parallel computing

A.6.3.0.4 Winning Together:

- The team wins when all cards are sorted correctly in the shared container
- All players must have returned their buffer cards to the container

A.7 Settings

- **Theme**: Toggle between light and dark mode
- **Signaling Server URL**: Configure a custom signaling server address for multiplayer (advanced; defaults to the hosted server)

A.8 Tips and Strategies

A.8.1 Sorting Strategies

1. **Divide and Conquer**: Use buffer slots to sort smaller subsets of cards, then merge them back

2. **Find Sorted Runs**: Identify cards that are already in order and build around them
3. **Minimize Moves**: Think before moving—unnecessary rearrangements waste time

A.8.2 Multiplayer Coordination

1. **Divide Responsibility**: Each player takes a value range (e.g., player 1 handles cards 1–25, player 2 handles 26–50)
2. **Use Barriers Strategically**: Initiate a barrier when all players have sorted their portions and need to merge
3. **Communicate Externally**: Use voice chat or messaging to coordinate since the game does not include built-in chat
4. **Main Thread Awareness**: When you are the main thread during a barrier, merge sorted chunks efficiently; other players must wait

A.9 Troubleshooting

A.9.1 Common Issues and Solutions

A.9.1.0.1 Game does not load in browser:

- Verify that your browser supports WebGL 2.0 (check `chrome://gpu` in Chrome)
- Ensure the hosting server sends correct COOP/COEP headers for `SharedArrayBuffer`
- Try clearing browser cache and reloading
- Try a different Chromium-based browser

A.9.1.0.2 Performance issues:

- Reduce the number of cards
- Close other browser tabs to free memory
- Ensure hardware acceleration is enabled in browser settings
- The desktop version generally performs better than the web version for large card counts

A.9.1.0.3 Cannot connect to multiplayer:

- Verify internet connection is active
- Check that both players are using the same game version
- Ensure the signaling server is reachable (default server requires internet access)
- Re-enter the lobby code carefully
- If using a custom signaling server, verify it is running and accessible

A.9.1.0.4 Multiplayer desynchronization:

- This can occur if network packets are lost or arrive out of order
- The host can use the "sync state" mechanism to push authoritative state to all clients
- As a last resort, restart the game session

A.10 Educational Use

A.10.1 For Students

- Play single-player mode first to understand the card sorting mechanics
- Observe how buffer slots help organize your sorting work (thread-local storage)
- In multiplayer, notice how coordination becomes harder with more players (communication overhead)
- Pay attention to the barrier mechanic—it directly mirrors `#pragma omp barrier`
- Reflect on how game concepts relate to OpenMP parallel programming

A.10.2 For Instructors

- Use as a pre-lecture activity to introduce HPC concepts before formal instruction
- The web deployment requires no installation—students can play immediately via a shared URL
- Facilitate post-gameplay discussion connecting game mechanics to parallel programming:

  - Shared container ↔ shared memory
  - Private buffers ↔ thread-local / private variables
  - Barrier mechanic ↔ `#pragma omp barrier`
  - Multiple players ↔ multiple threads
  - Single-player ↔ sequential execution baseline
- Use completion metrics to discuss relative effort of parallel coordination vs. sequential sorting

A.11 Support and Resources

- **Game Repository**: https://github.com/Siponek/hpc-sorting-serious-game
- **Thesis Repository**: https://github.com/Siponek/HPC-thesis
- **Issue Tracker**: Report bugs and request features via GitHub Issues
- **License**: MIT license—contributions are welcome

Appendix B

Code Listings

B.1 Source Code Repository

The complete source code for the HPC Sorting Serious Game is available as an open-source project:

- **Game Repository**: https://github.com/Siponek/hpc-sorting-serious-game
- **GDSync Fork**: https://github.com/Siponek/GD-Sync-fork (web-export fix; upstream PR #108)
- **License**: MIT License
- **Language**: GDScript (Godot Engine 4.x)

B.2 Project Structure

The project follows Godot Engine conventions. The listing below shows the key directories and files relevant to this thesis:

```
hpc-sorting-serious-game/
+-- project.godot                 # Project configuration
+-- scene_manager.gd              # Global scene transition autoload
+-- settings.gd                   # Global settings autoload
+-- theme_manager.gd              # Dark/light theme autoload
+-- export_presets.cfg            # Export presets (Web, Windows, Android)
+-- scenes/
|   +-- CardScene/
|   |   +-- scripts/
|   |   |   +-- card_manager.gd  # Base card manager (705 lines)
|   |   +-- card_scene.tscn      # Card prefab scene
|   +-- MainMenuScene/
|   |   +-- menu_scene.tscn      # Main menu
|   |   +-- Singleplayer/
```

```
|   |   |   +-- singleplayer_options.gd
|   |   +-- Multiplayer/
|   |       +-- multiplayer_options.gd
|   +-- Singleplayer/
|   |   +-- singleplayer_scene.tscn
|   +-- Multiplayer/
|   |   +-- connection_manager.gd       # GDSync signal wiring (348 lines)
|   |   +-- MultiplayerGame/
|   |   |   +-- multiplayer_card_manager.gd  # Extends card_manager (1143 lines)
|   |   |   +-- barrier_manager.gd           # Barrier state machine (113 lines)
|   |   +-- Lobby/
|   |   |   +-- multiplayer_lobby.gd
|   |   +-- GDSyncWebPatch/
|   |       +-- gdsync_web_patch.gd          # Web platform patch (67 lines)
|   |       +-- local_server_signaling.gd    # HTTP-based LocalServer (~600 lines)
|   |       +-- signaling_client.gd          # HTTP/SSE signaling client
+-- addons/
|   +-- GD-Sync/                  # GDSync plugin (git submodule, forked)
|   +-- var_tree/                 # Runtime variable inspector
|   +-- toastparty/              # Toast notification plugin
|   +-- scene-selector/          # Scene selector plugin
+-- lib/
|   +-- logger/                  # Structured logging library
|   +-- feature_flag_handlers/   # Feature flag utilities
+-- res/
|   +-- algorithms/              # Algorithm description resources
|   +-- dark_theme.tres          # Dark theme resource
|   +-- menu_theme.tres          # Menu theme resource
|   +-- shaders/                 # Visual effect shaders
+-- signaling-server/
|   +-- server.py                # Python aiohttp signaling server
|   +-- server/                  # Server module package
|   +-- tests/                   # Server test suite
+-- exports/
|   +-- web-export/              # Web build output
|   +-- certs/                   # TLS certificates for local dev
```

B.3 Key Components

B.3.1 Card System

- `card_manager.gd`: Base class handling card generation, drag-and-drop, buffer slot management, sorting validation (`check_sorting_order()`), buffer contiguity checking, and swap animations

- `multiplayer_card_manager.gd`: Extends `card_manager.gd`; adds `CardState` serialization, GDSync function exposure, network broadcast/reconciliation, barrier integration, and visibility management for multi-player buffers

- `barrier_manager.gd`: Three-state machine (`RUNNING` → `WAITING_AT_BARRIER` → `BARRIER_ACTIVE`) implementing `#pragma omp barrier` semantics with main-thread selection

B.3.2 Networking and Multiplayer

- `connection_manager.gd`: Autoload that wires GDSync signals (`connected`, `lobby_created`, `client_joined`, etc.) to game-level events; maintains typed player map

- `gdsync_web_patch.gd`: Autoload that detects `OS.has_feature("web")` and replaces GDSync's native `LocalServer` with the HTTP/SSE-based `local_server_signalin`

- `local_server_signaling.gd`: HTTP-based replacement for GDSync's `LocalServer`; preserves the original API contract while using the signaling server for packet relay

- `signaling_client.gd`: HTTP/SSE client for lobby management and real-time event streaming

B.3.3 Infrastructure

- `scene_manager.gd`: Global autoload managing scene transitions with loading screen

- `settings.gd`: Global autoload storing game configuration (card count, buffer count, barrier mode, multiplayer flag)

- `theme_manager.gd`: Dark/light theme switching

- `server.py`: Python signaling server built on `aiohttp`; handles lobby create/join, SSE event streaming, and game packet relay

B.4 Configuration

B.4.1 Autoloads

The following singletons are registered in `project.godot` and available globally:

- **Settings**: Game parameters (card count, buffer count, barrier mode)

- **SceneManager**: Scene transitions
- **ThemeManager**: Visual theme control
- **ConnectionManager**: Multiplayer session state
- **GDSync**: Networking framework (third-party plugin)
- **GDSyncWebPatch**: Web-export compatibility layer
- **ToastParty**: User notification toasts

B.4.2 Web Export Settings

Key settings in the `export_presets.cfg` Web preset:

- **Export Format**: WebAssembly (WASM)
- **Thread Support**: Requires `SharedArrayBuffer` (COOP/COEP headers)
- **Canvas Resize Policy**: Responsive
- **Output**: `exports/web-export/` directory

B.5 Building and Running

B.5.1 Development Setup

1. Install Godot Engine 4.x
2. Clone the repository with submodules: `git clone -recursive`
3. Open `project.godot` in the Godot Editor
4. Press F5 to run locally (desktop mode)

B.5.2 Web Export

1. In Godot Editor: Project → Export
2. Select the "Web" preset
3. Click "Export Project"
4. Deploy the contents of `exports/web-export/` to a web server that sets the required COOP/COEP headers

The project includes a `justfile` with automation targets for common tasks (formatting, linting, export, server management).

B.5.3 Signaling Server

The HTTP/SSE signaling server is in the `signaling-server/` directory:

1. Install Python 3.10+ and dependencies: `pip install aiohttp aiohttp-cors`
2. Run: `python server.py`
3. Default port: 8765 (configurable)

See Appendix C for the signaling server API reference.

Appendix C

API Documentation

C.1 Introduction

This appendix documents the APIs and interfaces of the HPC Sorting Serious Game, focusing on the signaling server REST/SSE endpoints and the key GDScript classes. The signaling server is the only custom server-side component; the game client is built in Godot Engine 4.x with GDScript.

C.2 Signaling Server API

The signaling server is a Python application built on `aiohttp`. It serves three route groups: HTTP session management, HTTP lobby management with SSE, and WebSocket signaling for WebRTC.

C.2.1 Session Endpoints

These endpoints manage rooms for WebRTC signaling.

Table C.1: Session management endpoints

Method	Path	Description
`POST`	`/session/host`	Create a new room. Body: `{channel, lobby_name, public, player_limit}`. Returns `{code, ws_url}`.
`POST`	`/session/join/{code}`	Join a room by code or lobby name. Returns `{code, ws_url}`.
`POST`	`/session/update/{code}`	Update room metadata (lobby name, public flag, player limit).
`POST`	`/session/players/{code}`	Update player count for a room.
`POST`	`/session/close/{code}`	Close a room and its associated lobby.

C.2.2 HTTP Lobby Endpoints (SSE)

These endpoints replace WebSocket-based lobby handling for browser compatibility. Real-time events are delivered via Server-Sent Events (SSE).

Table C.2: HTTP lobby endpoints

Method	Path	Description
`POST`	`/api/lobby/connect`	Connect and receive a peer ID. Optional body: `{client_id}` to specify a GDSync client ID.
`POST`	`/api/lobby/disconnect`	Disconnect from the server.
`POST`	`/api/lobby/create`	Create a new lobby. Body: `{lobby_name, public, player_limit}`. Returns `{code, lobby}`.
`POST`	`/api/lobby/join`	Join a lobby. Body: `{code}` or `{lobby_name}`.
`POST`	`/api/lobby/leave`	Leave the current lobby.
`GET`	`/api/lobby/list`	List public lobbies.
`POST`	`/api/lobby/broadcast`	Broadcast a game packet to lobby peers. Body: `{data}`.
`GET`	`/api/lobby/events`	SSE event stream. Query: `?peer_id={id}`.

C.2.3 SSE Event Types

The SSE stream (`GET /api/lobby/events`) delivers the following event types:

Table C.3: SSE event types

Event	Description
`welcome`	Initial connection acknowledgement with peer ID
`peer_joined`	A new peer joined the lobby
`peer_left`	A peer left the lobby
`lobby_joined`	Confirmation that this peer joined a lobby
`lobby_closed`	The lobby was closed (with reason)
`game_packet`	A game data packet relayed from another peer
`heartbeat`	Keep-alive ping
`error`	Error notification
`server_shutdown`	Server is shutting down

C.2.4 WebSocket Endpoints

These provide WebRTC signaling (for native desktop clients) and legacy WebSocket lobby handling:

Table C.4: WebSocket endpoints

Path	Description
`/ws/{code}`	WebRTC signaling channel. Exchanges ICE candidates, SDP offers/answers between peers in a room.
`/lobby`	WebSocket-based lobby management (native client usage; browser clients use HTTP+SSE instead).

C.2.5 Utility Endpoints

Table C.5: Utility endpoints

Method	Path	Description
`GET`	`/health`	Health check. Returns room, lobby, and peer counts.
`GET`	`/rooms`	List all active rooms.
`GET`	`/lobbies`	List all lobbies with peer details.
`GET`	`/api/server/info`	Server information (version, uptime, configuration).
`GET`	`/`	Welcome page / root info.

C.2.6 Error Codes

All error responses follow the format `{"success": false, "error": "<code>", "message": "..."}`.

C.3 Server Data Models

The signaling server uses two primary data models:

C.3.0.0.1 Peer: Represents a connected player. Fields: `peer_id` (int), `ws` (WebSocket connection, optional), `sse_queue` (asyncio Queue for SSE events, optional), `player_data` (dict), `lobby_code` (str, optional). A peer uses either WebSocket *or* SSE, not both—this dual-transport design enables both native and browser clients.

Table C.6: Signaling server error codes

Code	Meaning
`LOBBY_NOT_FOUND`	Specified lobby does not exist
`LOBBY_CLOSED`	Lobby has been closed
`LOBBY_FULL`	Lobby has reached player limit
`ALREADY_IN_LOBBY`	Peer is already in a lobby
`NOT_IN_LOBBY`	Peer is not in any lobby
`ROOM_NOT_FOUND`	Specified room does not exist
`PEER_NOT_FOUND`	Specified peer ID not found
`PEER_ID_IN_USE`	Requested peer ID is already taken
`UNKNOWN_COMMAND`	Unrecognized message type
`INVALID_JSON`	Malformed JSON in request body

C.3.0.0.2 Lobby: Represents a game lobby. Fields: `code` (str), `name` (str), `host_id` (int), `public` (bool), `player_limit` (int), `open` (bool), `created_at` (ISO timestamp), `peers` (dict of Peer).

C.4 GDScript Game Classes

This section summarizes the key GDScript classes. Full source code is available in the repository (see Appendix B.1).

C.4.1 CardManager (card_manager.gd)

Base class managing all card logic for single-player mode.

C.4.1.0.1 Key Properties:

- `num_cards: int` — Number of cards in the game
- `cards_array: Array` — All card node instances
- `sorted_all: Array` — Sorted reference array (target ordering)
- `slots: Array` — Buffer slot instances
- `card_container: HBoxContainer` — Shared card container (scene tree node)

C.4.1.0.2 Key Methods:

- `check_sorting_order() -> bool` — Validates ascending order of cards in container

- `check_player_buffer_contiguity() -> bool` — Validates buffer cards form a contiguous sorted subsequence
- `generate_completed_card_array(values, prefix) -> Array` — Creates card instances from value array
- `create_buffer_slots() -> Array` — Instantiates player buffer slots
- `animate_card_swap(card_a, card_b)` — Animated card position swap with tween

C.4.2 MultiplayerCardManager (multiplayer_card_manager.gd)

Extends `CardManager` for multiplayer mode.

C.4.2.0.1 Inner Class — CardState:

- Serializable state object with `to_dict()` and `from_dict()` methods
- Fields: `value`, `index`, `original_index`, `in_container`, `in_buffer`, `buffer_owner`

C.4.2.0.2 Key Methods (GDSync-exposed):

- `sync_complete_game_state(state_data)` — Full state reconciliation
- `sync_card_moved(value, from_idx, to_idx, client_id)` — Card repositioning
- `sync_card_entered_buffer(value, player_id)` — Card enters private buffer
- `sync_card_left_buffer(value, player_id, new_idx)` — Card leaves buffer
- `barrier_thread_reached(player_id)` — Player reached barrier
- `barrier_activate()` — All threads at barrier; activate
- `barrier_release()` — Release barrier; resume normal play

C.4.3 BarrierManager (barrier_manager.gd)

Implements barrier synchronization state machine.

C.4.3.0.1 States: `RUNNING` → `WAITING_AT_BARRIER` → `BARRIER_ACTIVE` → `RUNNING`.

C.4.3.0.2 Key Methods:

- `enter_waiting_state(initiator_id, all_player_ids)` — Transition to waiting state
- `activate_barrier()` — Transition to active state (all threads arrived)
- `release_barrier()` — Reset and return to running state
- `determine_main_thread(first_id, all_ids) -> int` — Select coordinator (first-to-reach or round-robin)

C.4.3.0.3 Signals:

- `barrier_state_changed(new_state)` — State transition notification
- `thread_reached_barrier(player_id)` — Player arrival notification
- `main_thread_assigned(player_id)` — Main thread selected

C.4.4 ConnectionManager (connection_manager.gd)

Autoload that wraps GDSync signals into game-level events.

C.4.4.0.1 Key Methods:

- `am_i_host() -> bool` — Whether local player is lobby host
- `get_my_client_id() -> int` — Local player's GDSync client ID
- `get_player_list() -> MultiplayerTypes.PlayersMap` — Current player map

C.4.4.0.2 Key Signals:

- `player_joined_lobby(player_id)` — Player entered lobby
- `player_list_updated(players)` — Player list changed
- `lobby_joined(lobby_name)` — Local player joined a lobby

C.5 GDSync Framework Usage

This project uses GDSync's explicit function exposure model. GDSync does *not* use annotations or decorators for RPC. Instead, functions are registered at runtime:

```
# Expose a function for remote calls
GDSync.expose_func(self.my_function)

# Call function on all peers
GDSync.call_func(self.my_function, [arg1, arg2])

# Call function on a specific peer
GDSync.call_func_on(target_peer_id, self.my_function, [arg1, arg2])

# Expose a variable for synchronization
GDSync.expose_var(self, "variable_name")
```

Listing C.1: GDSync function exposure (actual API)

C.5.0.0.1 Key GDSync Signals:

- `GDSync.connected` — Successfully connected to server
- `GDSync.connection_failed` — Connection attempt failed
- `GDSync.lobby_created` — Lobby created (host)
- `GDSync.lobby_joined` — Joined an existing lobby
- `GDSync.client_joined` — A new client joined the lobby
- `GDSync.client_left` — A client left the lobby
- `GDSync.lobbies_received` — Lobby list received

For protected mode details and the web-export compatibility layer, see Chapter 6, Section 6.3.1 and Chapter 5, Section 5.6.

Appendix D

Study Materials

D.1 Introduction

This appendix provides educational materials to help students and instructors connect the HPC Sorting Serious Game to formal parallel computing concepts. It includes learning objectives, discussion questions, exercises, and connections to OpenMP programming and sequential vs. parallel execution.

D.2 Learning Objectives

D.2.1 Foundational Concepts

After playing the game, students should be able to:

1. **Define Parallelism**: Explain what it means to perform multiple tasks at the same time
2. **Understand Teamwork Benefits**: Describe how working together can finish tasks faster
3. **Recognize Coordination Challenges**: Explain that working together requires communication and planning
4. **Appreciate Fair Work Distribution**: Understand why it's important that everyone has about the same amount of work
5. **Identify When Teamwork Helps**: Recognize that some tasks benefit from multiple workers more than others

D.2.2 Sequential Execution Concepts (Single-Player Mode)

Students should understand:

1. A single process executes all operations sequentially

2. No parallelism or concurrency
3. Simple but potentially slower for large datasets
4. Baseline for comparison with parallel execution
5. All data is accessible directly without coordination

D.2.3 OpenMP Concepts (Multiplayer Mode)

Students should understand:

1. Multiple threads share a common memory space
2. Each thread can have private (local) data in buffers
3. Threads work collaboratively on shared data
4. Synchronization occurs through shared container access
5. Work is divided among threads for parallel execution
6. Final merge combines results from parallel execution

D.3 Discussion Questions

D.3.1 Pre-Game Questions

Use these questions before gameplay to activate prior knowledge:

1. Have you ever worked on a group project? How did you divide the work?
2. What challenges arise when multiple people work on the same task simultaneously?
3. How is sorting a deck of cards different when one person does it versus multiple people?
4. Can you think of tasks that benefit from having multiple workers? Which tasks don't?

D.3.2 Post-Game Discussion: Single-Player Mode

After single-player gameplay:

1. How did you use the buffer zones? Why were they helpful?
2. What strategy did you use to sort the cards efficiently?
3. How would your strategy change with 10 cards versus 200 cards?
4. If you could have multiple versions of yourself working simultaneously, how would you coordinate?
5. What parallels can you draw between the buffers and computer memory?

D.3.3 Post-Game Discussion: Multiplayer Mode

After multiplayer gameplay:

1. How did you communicate and coordinate with other players?
2. What happened when two players tried to work on the same cards?
3. Did you notice delays when passing cards between players? Why might that occur?
4. How did you divide the work? Was it balanced?
5. What would happen if one player had many more cards than others?
6. How is this different from single-player mode?

D.3.4 Connecting to HPC Concepts

Bridge from game to formal concepts:

1. How does single-player mode relate to sequential (single-threaded) execution?
2. How does multiplayer mode relate to shared-memory parallel programming (OpenMP)?
3. What real-world computing problems benefit from parallelism?
4. What role does the shared container play compared to private buffers?
5. When might coordination overhead outweigh parallelism benefits?

D.4 Classroom Activities

D.4.1 Activity 1: Strategy Competition

D.4.1.0.1 Objective: Compare different ways to sort cards and see which works best.

D.4.1.0.2 Instructions:

1. Divide class into teams of 2–3 students
2. Each team plays the game with the same settings (e.g., 50 cards, 3 players)
3. Teams write down their strategy: How will you divide the work? Will you help each other?
4. Compare completion times between teams
5. Discuss which strategies worked well and why

D.4.1.0.3 Discussion:

- Which strategy was fastest? Why do you think it worked better?
- Did teams that divided work evenly finish faster?
- What problems did your team face during the game?
- If you played again, what would you do differently?

D.4.2 Activity 2: Does More Players Mean Faster?

D.4.2.0.1 Objective: Investigate how adding more players affects completion time.

D.4.2.0.2 Instructions:

1. Students play multiplayer mode with different numbers of players:
   - 2 players, 50 cards
   - 3 players, 50 cards
   - 4 players, 50 cards
2. Record completion times for each configuration
3. Create a simple chart showing number of players vs. time
4. Discuss the results

D.4.2.0.3 Analysis Questions:

- Does doubling the number of players cut the time in half?
- What makes it hard to keep getting faster as you add more players?
- Can you think of tasks where adding more people doesn't help much?

D.4.3 Activity 3: Working Together vs. Working Separately

D.4.3.0.1 Objective: Understand when it's better to work together versus work separately.

D.4.3.0.2 Instructions:

1. Play multiplayer mode with different strategies:
   - **Strategy A**: Each player sorts their own cards independently, then combine at the end
   - **Strategy B**: Players frequently pass cards to each other to help balance the work
2. Record your time and count how many times cards were passed between players
3. Compare which strategy was faster

D.4.3.0.3 Discussion:

- Which strategy was faster?
- Does passing cards between players take time? How much?
- When is it worth the time to pass cards to another player?
- Can you think of real-world situations where coordination takes more time than it saves?

D.5 Exercises

These exercises are designed for students and learners with no prior HPC experience. They focus on understanding parallel computing concepts through gameplay rather than technical implementation.

D.5.1 Exercise 1: Comparing Single and Multiple Players

D.5.1.0.1 Task: Play the game twice with the same number of cards (e.g., 30 cards):

- First time: Single-player mode
- Second time: Multiplayer mode with 2-3 friends

D.5.1.0.2 Questions:

1. Which mode finished faster?
2. Was multiplayer always faster? If not, why not?
3. What made working together harder than working alone?
4. If you added more players, would it keep getting faster forever? Why or why not?

D.5.2 Exercise 2: Finding the Best Team Size

D.5.2.0.1 Task: Try sorting 50 cards with different numbers of players and record your time:

- 1 player (you alone)
- 2 players
- 3 players
- 4 players

D.5.2.0.2 Questions:

1. Which team size was fastest?
2. Did more players always mean faster sorting?
3. What problems happened when you had too many players?
4. Can you think of real-world situations where having too many helpers makes things slower?

D.5.3 Exercise 3: Dividing the Work

D.5.3.0.1 Task: Play multiplayer mode and try two different strategies:

- **Strategy A**: Each player takes an equal number of cards at the start and sorts them
- **Strategy B**: Players help whoever has the most cards remaining

D.5.3.0.2 Questions:

1. Which strategy felt easier to coordinate?
2. Which strategy finished faster?
3. What happens if one player is much faster than the others?
4. How does this relate to group projects in school?

D.6 Conceptual Mappings

D.6.1 Game to OpenMP

Table D.1: Detailed game-to-OpenMP concept mapping

Game Element	HPC/OpenMP Concept	Code Example
Single-player mode	Sequential execution	Single-threaded program
Multiplayer session	Parallel region	`#pragma omp parallel`
Main card container	Shared array	`shared(array)`
Private buffer zones	Thread-private storage	`private(buffer)`
Players working independently	Independent threads	No explicit synchronization
Sorting within buffer	Local computation	Serial code in parallel region
Merging back to main	Critical section	`#pragma omp critical`
Barrier mechanic	Barrier synchronization	`#pragma omp barrier`
Main thread during barrier	Master thread	`#pragma omp master`

D.7 Assessment Rubric

For instructors wishing to grade student understanding:

D.7.1 Conceptual Understanding (40 points)

- (10 pts) Can identify shared-memory vs. distributed-memory paradigms
- (10 pts) Understands role of communication in parallel performance
- (10 pts) Can explain speedup and scalability concepts
- (10 pts) Recognizes load balancing importance

D.7.2 Practical Application (30 points)

- (15 pts) Designs reasonable parallel sorting strategy using in-game tools
- (15 pts) Demonstrates effective use of buffers, barriers, and coordination during gameplay

D.7.3 Analysis and Reflection (30 points)

- (10 pts) Analyzes performance data from experiments
- (10 pts) Connects game experience to real HPC systems
- (10 pts) Reflects on limitations and trade-offs

D.8 Additional Resources

D.8.1 Recommended Readings

1. Pacheco, P. (2011). *An Introduction to Parallel Programming.* Morgan Kaufmann.
2. OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface.*
3. Chapman, B., Jost, G., & Van Der Pas, R. (2007). *Using OpenMP: Portable Shared Memory Parallel Programming.* MIT Press.

D.8.2 Online Resources

- OpenMP Tutorials: `https://www.openmp.org/resources/tutorials-articles/`
- Parallel Computing Course Materials: `https://www.cs.cmu.edu/ 15418/`
- Lawrence Livermore National Lab OpenMP Tutorial: `https://hpc.llnl.gov/tuts/openMP/`

D.8.3 Hands-On Practice

- Try the game with different configurations
- Implement actual parallel sorting code
- Profile and optimize parallel programs
- Experiment with HPC clusters (if available)

D.9 Tool and Framework Reference Links

This section lists direct links to core tools, frameworks, and plugins referenced in the thesis. These links are also represented in the bibliography entries used in Chapters 3 and 5.

Table D.2: External tools and framework links used in this thesis

Tool	Category	URL
Godot Engine	Game engine	https://godotengine.org/
GDSync (upstream)	Multiplayer framework	https://www.gd-sync.com/
GDSync (project fork)	Multiplayer framework fork	https://github.com/Siponek/GD-Sync-fork
ToastParty	Godot plugin	https://godotassetlibrary.com/asset/PcSgaA/toast-party
VarTree	Godot plugin	https://godotassetlibrary.com/asset/PUudIL/vartree
Const Generator	Godot plugin	https://godotassetlibrary.com/asset/DrA6Bt/const-generator
Python	Language/runtime	https://www.python.org/
aiohttp	Python web framework	https://docs.aiohttp.org/
Ruff	Linter	https://docs.astral.sh/ruff/
just	Task runner	https://just.systems/
mkcert	Local TLS utility	https://github.com/FiloSottile/mkcert
Chocolatey	Package manager	https://chocolatey.org/
Git	Version control	https://git-scm.com/
GitHub	Collaboration hosting	https://github.com/

D.10 Instructor Notes

D.10.1 Integration with Curriculum

D.10.1.0.1 As Pre-Lecture Activity (15–20 minutes):

- Students play game before formal HPC lecture
- Generates curiosity and questions
- Provides concrete reference points for abstract concepts
- Follow with discussion connecting gameplay to course material

D.10.1.0.2 As Lab Exercise (50–90 minutes):

- Structured activities with data collection
- Analysis of performance metrics
- Written reflection connecting to theory
- Implementation of actual parallel code based on strategies

D.10.1.0.3 As Homework/Assessment:

- Students play independently, record observations
- Answer reflection questions
- Design parallel algorithms inspired by gameplay
- Optional: Implement and benchmark solutions

D.10.2 Common Student Misconceptions

Be prepared to address:

1. **"More processes/threads always means faster"**: Discuss communication overhead and Amdahl's Law
2. **"Shared memory means no synchronization needed"**: Explain race conditions and critical sections
3. **"Message passing is always slower"**: Discuss scalability benefits of distributed memory
4. **"The game perfectly simulates HPC"**: Clarify simplifications and abstractions